

```
# ===== FILE: backend/config.py =====

from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):

    # DeepSeek 配置既可以来自项目根目录的 .env，也可以由运行时设置覆盖。

    # 默认不填写密钥，避免源码和安装包意外携带用户的私密凭据。

    deepseek_api_key: str = ""

    deepseek_api_base: str = "https://api.deepseek.com"

    deepseek_model: str = "deepseek-chat"

    deepseek_mock: bool = False

    database_url: str = "sqlite:///./app.db"

    upload_dir: str = "uploads"

    frontend_origin: str = "http://localhost:5173"

    # TTS 配置统一描述浏览器、Kokoro、Sherpa 和兼容 HTTP 服务。

    # tts_device 使用 auto 时由后端探测 CUDA，无法使用 GPU 才回退 CPU。

    tts_enabled: bool = True

    tts_api_url: str = "http://127.0.0.1:9977/tts"

    tts_api_style: str = "kokoro"

    tts_voice: str = "zf_xiaoxiao"

    tts_english_voice: str = "af_heart"

    tts_model: str = "kokoro-82m"

    tts_lang_code: str = "z"

    tts_device: str = "auto"

    tts_speed: float = 0.8

    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

@lru_cache

def get_settings() -> Settings:

    # 配置对象在进程内只创建一次，保证所有路由读取到一致的环境配置。

    # 网页中可修改的设置另存于 runtime_settings.json，不依赖重启后端。

    return Settings()

# ===== FILE: backend/database.py =====

from sqlalchemy import create_engine

from sqlalchemy import inspect, text

from sqlalchemy.orm import DeclarativeBase, sessionmaker

from config import get_settings

settings = get_settings()

# SQLite 默认限制连接只能在创建它的线程中使用。FastAPI 会在线程池中执行

# 同步依赖，因此本地 SQLite 模式需要关闭该限制；换成其他数据库时不传此参数。

engine = create_engine(

    settings.database_url,

    connect_args={"check_same_thread": False} if settings.database_url.startswith("sqlite") else {},

)

SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)

class Base(DeclarativeBase):

    pass

def get_db():

    # 每个请求获得独立会话，并在响应结束后可靠关闭连接。

    # 路由只负责 commit 业务事务，不需要重复处理连接释放。

    db = SessionLocal()

    try:
```

```
        yield db
    finally:
        db.close()
def init_db():
    # 延迟导入模型，确保 SQLAlchemy 已收集全部表定义后再建表。
    import models # noqa: F401
    Base.metadata.create_all(bind=engine)
    # MVP 不引入完整迁移框架；仅对历史数据库补充向后兼容的新字段。
    _apply_lightweight_migrations()
def _apply_lightweight_migrations():
    # 先检查表和字段是否存在，使迁移可以在每次启动时幂等执行。
    inspector = inspect(engine)
    table_names = set(inspector.get_table_names())
    if "books" not in table_names or "concept_cards" not in table_names:
        return
    book_columns = {column["name"] for column in inspector.get_columns("books")}
    card_columns = {column["name"] for column in inspector.get_columns("concept_cards")}
    statements = []
    if "generation_cursor" not in book_columns:
        statements.append("ALTER TABLE books ADD COLUMN generation_cursor INTEGER DEFAULT 0")
    if "section_index" not in card_columns:
        statements.append("ALTER TABLE concept_cards ADD COLUMN section_index INTEGER DEFAULT 0")
    if "card_type" not in card_columns:
        statements.append("ALTER TABLE concept_cards ADD COLUMN card_type VARCHAR(30) DEFAULT 'concept'")
    if statements:
        # engine.begin 会将本轮 ALTER TABLE 放入同一个提交范围，发生异常时回滚。
        with engine.begin() as conn:
            for statement in statements:
                conn.execute(text(statement))
# ===== FILE: backend/desktop_launcher.py =====
from __future__ import annotations
import os
import socket
import sys
import threading
import time
import webbrowser
from pathlib import Path
import uvicorn
def _app_root() -> Path:
    if getattr(sys, "frozen", False):
        return Path(getattr(sys, "_MEIPASS", Path(sys.executable).resolve().parent))
    return Path(__file__).resolve().parents[1]
def _find_frontend_dist(root: Path -> Path | None:
    executable_dir = Path(sys.executable).resolve().parent
    candidates = [
        root / "frontend" / "dist",
        executable_dir / "_internal" / "frontend" / "dist",
        executable_dir / "frontend" / "dist",
    ]
```

```
for candidate in candidates:
    if candidate.exists():
        return candidate
return None

def _prepare_environment() -> None:
    root = _app_root()
    backend_dir = root / "backend"
    if str(backend_dir) not in sys.path:
        sys.path.insert(0, str(backend_dir))
    frontend_dist = _find_frontend_dist(root)
    if frontend_dist:
        os.environ["BOOK_CONCEPT_FRONTEND_DIST"] = str(frontend_dist)
    os.chdir(backend_dir)
    os.environ.setdefault("PYTHONUTF8", "1")
    os.environ.setdefault("PYTHONIOENCODING", "utf-8")

def _open_browser(url: str) -> None:
    time.sleep(2)
    webbrowser.open(url)

def _find_available_port(start: int = 8000, attempts: int = 20) -> int:
    for port in range(start, start + attempts):
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
            try:
                sock.bind(("127.0.0.1", port))
            except OSError:
                continue
        return port
    raise RuntimeError(f"无法找到可用端口：{start}-{start + attempts - 1}")

def main() -> None:
    if "--kokoro-worker" in sys.argv:
        _prepare_environment()
        from services.kokoro_worker import main as worker_main
        worker_main()
        return
    _prepare_environment()
    from main import app
    port = _find_available_port()
    url = f"http://127.0.0.1:{port}"
    threading.Thread(target=_open_browser, args=(url,), daemon=True).start()
    uvicorn.run(app, host="127.0.0.1", port=port, log_level="info")

if __name__ == "__main__":
    main()

# ===== FILE: backend/init_db.py =====

from database import init_db

if __name__ == "__main__":
    init_db()
    print("Database initialized.")

# ===== FILE: backend/main.py =====

import sys
import os
```

```
from pathlib import Path
from fastapi import FastAPI
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from fastapi.middleware.cors import CORSMiddleware
from config import get_settings
from database import init_db
from routers import books, cards, chat, settings as settings_router, tts, upload
from services.kokoro_manager import start_kokoro_if_enabled
settings = get_settings()
app = FastAPI(title="Book Concept App API", version="0.1.0")
# 开发模式下前端端口可能变化, 因此同时允许配置来源和本机动态端口。
# API Key 只由后端保存与调用, 浏览器跨域访问不会获得密钥原文。
app.add_middleware(
    CORSMiddleware,
    allow_origins=[settings.frontend_origin, "http://127.0.0.1:5173"],
    allow_origin_regex=r"https?://(localhost|127\.\.0\.\.1):\d+",
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
@app.on_event("startup")
def on_startup():
    # 后端启动时先保证数据库结构可用, 再按当前设置按需拉起本地语音服务。
    init_db()
    start_kokoro_if_enabled()
@app.get("/api/health")
def health():
    return {"status": "ok"}
app.include_router(upload.router)
app.include_router(books.router)
app.include_router(cards.router)
app.include_router(chat.router)
app.include_router(settings_router.router)
app.include_router(tts.router)
def _find_frontend_dist() -> Path:
    # 查找顺序同时覆盖源码开发目录、PyInstaller 临时解包目录和安装目录。
    # 这样同一套 FastAPI 入口既能独立运行, 也能在桌面安装包中托管前端。
    configured = os.getenv("BOOK_CONCEPT_FRONTEND_DIST")
    if configured:
        configured_path = Path(configured)
        if configured_path.exists():
            return configured_path
    candidates = [
        Path(__file__).resolve().parents[1] / "frontend" / "dist",
    ]
    bundle_root = getattr(sys, "_MEIPASS", None)
    if bundle_root:
        candidates.append(Path(bundle_root) / "frontend" / "dist")
    executable_dir = Path(sys.executable).resolve().parent
```

```

candidates.extend(
    [
        executable_dir / "_internal" / "frontend" / "dist",
        executable_dir / "frontend" / "dist",
    ]
)

for candidate in candidates:
    if candidate.exists():
        return candidate
return candidates[0]

FRONTEND_DIST = _find_frontend_dist()

if FRONTEND_DIST.exists():
    # Vite 构建后的静态资源使用 /assets 路径，单独挂载可避免被 SPA 回退接管。
    assets_dir = FRONTEND_DIST / "assets"
    if assets_dir.exists():
        app.mount("/assets", StaticFiles(directory=assets_dir), name="assets")

@app.get("/")
def root():
    # 已构建前端时直接返回网页首页；开发模式未构建时返回后端诊断信息。
    index_file = FRONTEND_DIST / "index.html"
    if index_file.exists():
        return FileResponse(index_file)
    return {
        "name": "Book Concept App API",
        "status": "ok",
        "docs": "/docs",
        "health": "/api/health",
        "frontend": settings.frontend_origin,
    }

@app.get("/{full_path:path}")
def frontend_fallback(full_path: str):
    # 浏览器刷新前端子路由时仍返回 index.html，由 React 接管页面状态。
    # API 未命中时必须保持 JSON 404，不能误返回 HTML 导致前端解析失败。
    if full_path.startswith("api/"):
        return {"detail": "Not Found"}
    index_file = FRONTEND_DIST / "index.html"
    if index_file.exists():
        return FileResponse(index_file)
    return {"detail": "Frontend build not found. Run npm run build first."}

# ===== FILE: backend/models.py =====

from datetime import datetime

from sqlalchemy import Boolean, DateTime, ForeignKey, Integer, String, Text

from sqlalchemy.orm import Mapped, mapped_column, relationship

from database import Base

class Book(Base):
    # Book 保存原始全文、解析后的小节 JSON，以及可断点续生成的游标。
    # generation_cursor 指向下一待生成小节，关闭网页后仍能从数据库恢复进度。
    __tablename__ = "books"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    title: Mapped[str] = mapped_column(String(255), index=True)
```

```
filename: Mapped[str] = mapped_column(String(255))
file_type: Mapped[str] = mapped_column(String(20))
raw_text: Mapped[str] = mapped_column(Text)
chunks_json: Mapped[str] = mapped_column(Text, default="[]")
generation_cursor: Mapped[int] = mapped_column(Integer, default=0)
created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
cards: Mapped[list["ConceptCard"]] = relationship(
    # 删除书籍时级联删除其知识卡片，避免留下失去归属的记录。
    back_populates="book",
    cascade="all, delete-orphan",
)

class ConceptCard(Base):
    # 每张卡片绑定书籍和小节序号。card_type 区分章节导览卡与普通概念卡。
    __tablename__ = "concept_cards"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    book_id: Mapped[int] = mapped_column(ForeignKey("books.id"), index=True)
    section_index: Mapped[int] = mapped_column(Integer, default=0, index=True)
    card_type: Mapped[str] = mapped_column(String(30), default="concept")
    chapter: Mapped[str] = mapped_column(String(255), default="")
    title: Mapped[str] = mapped_column(String(255), index=True)
    source_text: Mapped[str] = mapped_column(Text)
    one_sentence: Mapped[str] = mapped_column(Text, default="")
    simple_explanation: Mapped[str] = mapped_column(Text, default="")
    fable: Mapped[str] = mapped_column(Text, default="")
    formula: Mapped[str] = mapped_column(Text, default="")
    formula_explanation: Mapped[str] = mapped_column(Text, default="")
    prerequisites: Mapped[str] = mapped_column(Text, default="[]")
    # SQLite 不依赖数据库专用数组类型，列表字段以 JSON 文本保存，API 输出时再还原。
    related_concepts: Mapped[str] = mapped_column(Text, default="[]")
    questions: Mapped[str] = mapped_column(Text, default="[]")
    is_favorite: Mapped[bool] = mapped_column(Boolean, default=False)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    book: Mapped[Book] = relationship(back_populates="cards")
    messages: Mapped[list["ChatMessage"]] = relationship(
        back_populates="card",
        cascade="all, delete-orphan",
    )

class ChatMessage(Base):
    # 追问记录绑定具体卡片，保证切换概念后不会混用另一张卡片的上下文。
    __tablename__ = "chat_messages"

    id: Mapped[int] = mapped_column(Integer, primary_key=True, index=True)
    card_id: Mapped[int] = mapped_column(ForeignKey("concept_cards.id"), index=True)
    role: Mapped[str] = mapped_column(String(20))
    content: Mapped[str] = mapped_column(Text)
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
    card: Mapped[ConceptCard] = relationship(back_populates="messages")

# ===== FILE: backend/routers/_init_.py =====

# ===== FILE: backend/routers/books.py =====

from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.orm import Session
```

```
from database import get_db

from models import Book, ConceptCard

from schemas import BookOut, GenerateCardsOut, OutlineItemOut

from services.card_generator import build_outline, generate_cards_for_book

from services.deepseek_client import DeepSeekNotConfiguredError

router = APIRouter(prefix="/api/books", tags=["books"])

@router.get("", response_model=list[BookOut])

def list_books(db: Session = Depends(get_db)):

    # 书籍表不冗余保存卡片总数，列表查询时按 book_id 统计，避免生成失败造成计数漂移。

    books = db.query(Book).order_by(Book.created_at.desc()).all()

    result = []

    for book in books:

        data = BookOut.model_validate(book)

        data.card_count = db.query(ConceptCard).filter(ConceptCard.book_id == book.id).count()

        result.append(data)

    return result

@router.get("/{book_id}", response_model=BookOut)

def get_book(book_id: int, db: Session = Depends(get_db)):

    book = db.get(Book, book_id)

    if not book:

        raise HTTPException(status_code=404, detail="书籍不存在。")

    data = BookOut.model_validate(book)

    data.card_count = db.query(ConceptCard).filter(ConceptCard.book_id == book.id).count()

    return data

@router.get("/{book_id}/outline", response_model=list[OutlineItemOut])

def get_outline(book_id: int, db: Session = Depends(get_db)):

    book = db.get(Book, book_id)

    if not book:

        raise HTTPException(status_code=404, detail="书籍不存在。")

    # 大纲同时返回每节生成状态和卡片数，供阅读页显示后台生成进度。

    cards = db.query(ConceptCard).filter(ConceptCard.book_id == book.id).all()

    return build_outline(book, cards)

@router.post("/{book_id}/generate-cards", response_model=GenerateCardsOut)

async def generate_cards(

    book_id: int,

    force: bool = Query(False, description="是否清空旧卡片并从头生成"),

    db: Session = Depends(get_db),

):

    book = db.get(Book, book_id)

    if not book:

        raise HTTPException(status_code=404, detail="书籍不存在。")

    # 单次请求只处理一个小节，既缩短 DeepSeek 等待时间，也让断点游标及时落库。

    try:

        generated, done, cursor, total_sections = await generate_cards_for_book(db, book, force=force)

    # 配置问题、上游服务问题和未知程序错误使用不同状态码，前端可以给出准确提示。

    except DeepSeekNotConfiguredError as exc:

        raise HTTPException(status_code=400, detail=str(exc)) from exc

    except RuntimeError as exc:

        raise HTTPException(status_code=502, detail=str(exc)) from exc

    except Exception as exc:
```

```
        raise HTTPException(status_code=500, detail=f"生成失败：{type(exc).__name__}: {exc}") from exc
    if done:
        message = "本书全部小节已生成完成。"
    elif generated == 0:
        message = "本次没有生成新卡片， 请点击生成或检查当前小节内容。"
    else:
        message = "本次生成完成， 可继续生成后续小节。"
    return GenerateCardsOut(
        book_id=book.id,
        generated=generated,
        message=message,
        done=done,
        cursor=cursor,
        total_sections=total_sections,
    )
# ===== FILE: backend/routers/cards.py =====
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from database import get_db
from models import ConceptCard
from schemas import ConceptCardOut, FavoriteOut
from services.card_generator import card_to_dict
router = APIRouter(prefix="/api", tags=["cards"])
@router.get("/books/{book_id}/cards", response_model=list[ConceptCardOut])
def list_cards(book_id: int, db: Session = Depends(get_db)):
    # 先按小节、再按数据库主键排序， 使新增后续小节时已有卡片顺序保持稳定。
    cards = (
        db.query(ConceptCard)
        .filter(ConceptCard.book_id == book_id)
        .order_by(ConceptCard.section_index.asc(), ConceptCard.id.asc())
        .all()
    )
    return [card_to_dict(card) for card in cards]
@router.post("/cards/{card_id}/favorite", response_model=FavoriteOut)
def toggle_favorite(card_id: int, db: Session = Depends(get_db)):
    card = db.get(ConceptCard, card_id)
    if not card:
        raise HTTPException(status_code=404, detail="知识卡片不存在")
    # 收藏状态直接持久化， 用户下次打开同一本书时无需依赖浏览器本地缓存。
    card.is_favorite = not card.is_favorite
    db.commit()
    return FavoriteOut(card_id=card.id, is_favorite=card.is_favorite)
# ===== FILE: backend/routers/chat.py =====
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from database import get_db
from models import ChatMessage, ConceptCard
from schemas import ChatIn, ChatMessageOut, ChatOut
from services.deepseek_client import DeepSeekClient, DeepSeekNotConfiguredError
router = APIRouter(prefix="/api/cards", tags=["chat"])
```



```
@router.get("/{card_id}/chat", response_model=list[ChatMessageOut])

def list_messages(card_id: int, db: Session = Depends(get_db)):
    # 按时间正序返回历史消息， 前端可直接从上到下渲染对话。
    return (
        db.query(ChatMessage)
        .filter(ChatMessage.card_id == card_id)
        .order_by(ChatMessage.created_at.asc())
        .all()
    )

@router.post("/{card_id}/chat", response_model=ChatOut)
async def ask_card(card_id: int, payload: ChatIn, db: Session = Depends(get_db)):
    card = db.get(ConceptCard, card_id)
    if not card:
        raise HTTPException(status_code=404, detail="知识卡片不存在")
    question = payload.question.strip()
    if not question:
        raise HTTPException(status_code=400, detail="问题不能为空")
    # DeepSeekClient 会自动携带当前卡片原文、解释和公式， 避免回答脱离本节上下文。
    try:
        answer = await DeepSeekClient().answer_question(card, question)
    except DeepSeekNotConfiguredError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    except RuntimeError as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc
    # 用户问题与模型回答在同一次事务中写入， 避免只保存半段对话。
    db.add(ChatMessage(card_id=card.id, role="user", content=question))
    db.add(ChatMessage(card_id=card.id, role="assistant", content=answer))
    db.commit()
    return ChatOut(answer=answer)

# ===== FILE: backend/routers/settings.py =====

from fastapi import APIRouter

from schemas import DeepSeekSettingsIn, DeepSeekSettingsOut, TTSSettingsIn, TTSSettingsOut
from services.runtime_settings import get_deepseek_config, get_tts_config, masked_key, update_runtime_settings

router = APIRouter(prefix="/api/settings", tags=["settings"])

@router.get("/deepseek", response_model=DeepSeekSettingsOut)
def get_deepseek_settings():
    config = get_deepseek_config()
    # 接口只返回掩码后的密钥， 设置页面能够确认已配置， 但无法读取完整 API Key。
    return DeepSeekSettingsOut(
        configured=bool(config["api_key"]),
        api_key_masked=masked_key(config["api_key"]),
        api_base=config["api_base"],
        model=config["model"],
        mock=config["mock"],
    )

@router.put("/deepseek", response_model=DeepSeekSettingsOut)
def update_deepseek_settings(payload: DeepSeekSettingsIn):
    updates = {
        "deepseek_api_base": payload.api_base,
        "deepseek_model": payload.model,
```

```
        "deepseek_mock": payload.mock,
    }

    # 密钥输入留空表示保留旧值，防止用户只调整模型名称时意外清空密钥。
    if payload.api_key and payload.api_key.strip():
        updates["deepseek_api_key"] = payload.api_key.strip()
    update_runtime_settings(updates)
    return get_deepseek_settings()

@router.get("/tts", response_model=TTSSettingsOut)
def get_tts_settings():
    config = get_tts_config()
    return TTSSettingsOut(**config)

@router.put("/tts", response_model=TTSSettingsOut)
def update_tts_settings(payload: TTSSettingsIn):
    # 所有可选字段由服务层过滤 None，因此可以只提交语速等单项设置。
    update_runtime_settings(
        {
            "tts_enabled": payload.enabled,
            "tts_api_url": payload.api_url,
            "tts_api_style": payload.api_style,
            "tts_voice": payload.voice,
            "tts_english_voice": payload.english_voice,
            "tts_model": payload.model,
            "tts_lang_code": payload.lang_code,
            "tts_device": payload.device,
            "tts_speed": payload.speed,
        }
    )
    return get_tts_settings()

# ===== FILE: backend/routers/tts.py =====

from fastapi import APIRouter, HTTPException, Response
from schemas import SpeechIn, TTSPreloadOut
from services.tts_client import TTSNotConfiguredError, preload_tts, synthesize_speech

router = APIRouter(prefix="/api/tts", tags=["tts"])

@router.post("/speech")
async def speech(payload: SpeechIn):
    text = payload.text.strip()
    if not text:
        raise HTTPException(status_code=400, detail="朗读文本不能为空。")

    # 服务层统一处理引擎选择、模型懒加载和磁盘缓存，路由只负责 HTTP 语义。
    try:
        audio, content_type, cache_hit = await synthesize_speech(text, speed=payload.speed)
    except TTSNotConfiguredError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    except RuntimeError as exc:
        raise HTTPException(status_code=502, detail=str(exc)) from exc

    # X-TTS-Cache 便于前端和诊断工具确认本次是否直接复用了已生成音频。
    return Response(content=audio, media_type=content_type, headers={"X-TTS-Cache": "HIT" if cache_hit else "MISS"})

@router.post("/preload", response_model=TTSPreloadOut)
async def preload():
    # 预加载只初始化模型，不合成正文；用户可在开始阅读前主动消除首次等待。
```

```
try:
    detail = await preload_tts()
except TTSTNotConfiguredError as exc:
    raise HTTPException(status_code=400, detail=str(exc)) from exc
except RuntimeError as exc:
    raise HTTPException(status_code=502, detail=str(exc)) from exc
return TTSPreloadOut(status="ok", detail=detail)
# ===== FILE: backend/routers/upload.py =====
import json
from pathlib import Path
from uuid import uuid4
from fastapi import APIRouter, Depends, File, HTTPException, UploadFile
from sqlalchemy.orm import Session
from config import get_settings
from database import get_db
from models import Book
from schemas import UploadOut
from services.concept_splitter import split_into_chunks
from services.file_parser import read_text_file, validate_book_file
router = APIRouter(prefix="/api", tags=["upload"])
@router.post("/upload", response_model=UploadOut)
async def upload_book(file: UploadFile = File(...), db: Session = Depends(get_db)):
    # 先依据扩展名拒绝不支持的格式， MVP 明确只接收 Markdown 与纯文本。
    try:
        file_type = validate_book_file(file.filename or "")
    except ValueError as exc:
        raise HTTPException(status_code=400, detail=str(exc)) from exc
    settings = get_settings()
    upload_dir = Path(settings.upload_dir)
    upload_dir.mkdir(parents=True, exist_ok=True)
    # 使用随机前缀避免同名书籍覆盖； Path.name 同时剥离客户端提交的目录部分。
    safe_name = f"{uuid4().hex}_{Path(file.filename or 'book').name}"
    path = upload_dir / safe_name
    path.write_bytes(await file.read())
    # 文件服务负责编码探测和换行统一， 后续切分逻辑始终处理规范化 Unicode 文本。
    raw_text = read_text_file(path)
    if not raw_text:
        raise HTTPException(status_code=400, detail="文件内容为空")
    # 上传阶段先完成确定性的章节切分并保存， 生成卡片时无需重复解析整本书。
    chunks = split_into_chunks(raw_text)
    title = Path(file.filename or "未命名书籍").stem
    book = Book(
        title=title,
        filename=file.filename or safe_name,
        file_type=file_type,
        raw_text=raw_text,
        chunks_json=json.dumps(chunks, ensure_ascii=False),
    )
    # 原文与切分结果同时落库， 后续生成、追问和恢复进度都以本地数据库为准。
    db.add(book)
```

```
        db.commit()

        db.refresh(book)

        return UploadOut(book_id=book.id, title=book.title, message="上传成功")

# ===== FILE: backend/schemas.py =====

from datetime import datetime

from pydantic import BaseModel

class BookOut(BaseModel):

    # 书籍列表仅暴露元数据和生成进度，不把体积较大的 raw_text 返回给浏览器。

    id: int

    title: str

    filename: str

    file_type: str

    created_at: datetime

    card_count: int = 0

    generation_cursor: int = 0

    model_config = {"from_attributes": True}

class UploadOut(BaseModel):

    book_id: int

    title: str

    message: str

class GenerateCardsOut(BaseModel):

    book_id: int

    generated: int

    message: str

    done: bool = False

    cursor: int = 0

    total_sections: int = 0

class ConceptCardOut(BaseModel):

    # 数据库中的三个 JSON 文本列表在服务层还原后，以强类型数组交给前端。

    id: int

    book_id: int

    section_index: int = 0

    card_type: str = "concept"

    chapter: str

    title: str

    source_text: str

    one_sentence: str

    simple_explanation: str

    fable: str

    formula: str

    formula_explanation: str

    prerequisites: list[str]

    related_concepts: list[str]

    questions: list[str]

    is_favorite: bool

    created_at: datetime

class FavoriteOut(BaseModel):

    card_id: int

    is_favorite: bool

class ChatIn(BaseModel):
```

```
    question: str

class ChatOut(BaseModel):

    answer: str

class ChatMessageOut(BaseModel):

    id: int

    card_id: int

    role: str

    content: str

    created_at: datetime

    model_config = {"from_attributes": True}

class DeepSeekSettingsIn(BaseModel):

    # 设置字段允许局部更新；None 表示本次请求不改变该项。

    api_key: str | None = None

    api_base: str | None = None

    model: str | None = None

    mock: bool | None = None

class DeepSeekSettingsOut(BaseModel):

    configured: bool

    api_key_masked: str

    api_base: str

    model: str

    mock: bool

class TTSSettingsIn(BaseModel):

    enabled: bool | None = None

    api_url: str | None = None

    api_style: str | None = None

    voice: str | None = None

    english_voice: str | None = None

    model: str | None = None

    lang_code: str | None = None

    device: str | None = None

    speed: float | None = None

class TTSSettingsOut(BaseModel):

    enabled: bool

    api_url: str

    api_style: str

    voice: str

    english_voice: str

    model: str

    lang_code: str

    device: str

    speed: float

class SpeechIn(BaseModel):

    # 单次朗读允许覆盖全局语速，便于用户调节后立即试听而无需重启模型。

    text: str

    speed: float | None = None

class TTSPreloadOut(BaseModel):

    status: str

    detail: str

class OutlineItemOut(BaseModel):
```

```
index: int
title: str
preview: str
generated: bool
card_count: int
# ===== FILE: backend/services/__init__.py =====
# ===== FILE: backend/services/card_generator.py =====
from __future__ import annotations
import json
import re
from sqlalchemy.orm import Session
from models import Book, ChatMessage, ConceptCard
from services.deepseek_client import DeepSeekClient
MAX_CARDS_PER_RUN = 20
MAX_SECTION_CHARS = 5200
MAX_SECTIONS_PER_RUN = 1
async def generate_cards_for_book(db: Session, book: Book, force: bool = False) -> tuple[int, bool, int, int]:
    # “ 从头生成 ” 必须同时删除旧卡片、旧追问并重置游标，避免新旧知识体系混排。
    if force:
        _delete_book_cards(db, book.id)
        book.generation_cursor = 0
        db.commit()
    # 小节列表来自上传时保存的切分结果；游标指向下一次应处理的位置。
    # 游标每处理完一节就提交， 因此网络中断或关闭网页后仍可继续生成。
    client = DeepSeekClient()
    sections = _load_sections(book)
    generated = 0
    cursor = max(book.generation_cursor or 0, 0)
    processed_sections = 0
    while cursor < len(sections) and generated < MAX_CARDS_PER_RUN and processed_sections < MAX_SECTIONS_PER_RUN:
        section = sections[cursor]
        # 目录、参考书、机构网址等前置页不适合作为概念卡， 识别后只推进游标。
        if _is_probably_front_matter(section["text"], section["chapter"]):
            cursor += 1
            book.generation_cursor = cursor
            db.commit()
            continue
        # 单次调用最多请求六张卡， 并确保整个请求永远不超过二十张上限。
        remaining = MAX_CARDS_PER_RUN - generated
        target_count = min(6, remaining)
        cards = await client.generate_cards_from_section(
            section_text=section["text"],
            section_title=section["chapter"],
            target_count=target_count,
            # 每节第一批卡必须含一张导览卡；继续生成同一节时不重复创建导览。
            include_overview=not _section_has_overview(db, book.id, cursor),
        )
        processed_sections += 1
        saved = 0
        for card_data in cards:
```

```
        if generated >= MAX_CARDS_PER_RUN:
            break
        card = _build_card(book.id, cursor, section["chapter"], section["text"], card_data)
        # 服务端再次过滤标题，阻止模型偶尔返回的网址、目录词或空标题进入数据库。
        if not _is_valid_card_title(card.title):
            continue
        db.add(card)
        generated += 1
        saved += 1
    # 不论本节实际保存几张有效卡，都记录处理结果，避免坏小节导致无限重试。
    cursor += 1
    book.generation_cursor = cursor
    db.commit()
    if saved == 0 and generated > 0:
        break
done = cursor >= len(sections)
return generated, done, cursor, len(sections)

def build_outline(book: Book, cards: list[ConceptCard] | None = None) -> list[dict]:
    # 大纲不依赖模型重新总结，而是复用稳定的小节切分，并聚合各节卡片数量。
    sections = _load_sections(book)
    card_counts: dict[int, int] = {}
    if cards:
        for card in cards:
            card_counts[card.section_index] = card_counts.get(card.section_index, 0) + 1
    return [
        {
            "index": index,
            "title": _clean_title(section["chapter"]),
            "preview": _preview(section["text"]),
            "generated": index < (book.generation_cursor or 0),
            "card_count": card_counts.get(index, 0),
        }
        for index, section in enumerate(sections)
        if not _is_probably_front_matter(section["text"], section["chapter"])
    ]

def card_to_dict(card: ConceptCard) -> dict:
    # ORM 内部以 JSON 字符串保存列表；API 输出前统一转为数组并容忍历史坏数据。
    return {
        "id": card.id,
        "book_id": card.book_id,
        "section_index": card.section_index,
        "card_type": card.card_type,
        "chapter": card.chapter,
        "title": card.title,
        "source_text": card.source_text,
        "one_sentence": card.one_sentence,
        "simple_explanation": card.simple_explanation,
        "fable": card.fable,
        "formula": card.formula,
        "formula_explanation": card.formula_explanation,
```

```
        "prerequisites": _loads_list(card.prerequisites),
        "related_concepts": _loads_list(card.related_concepts),
        "questions": _loads_list(card.questions),
        "is_favorite": card.is_favorite,
        "created_at": card.created_at,
    }

def _load_sections(book: Book) -> list[dict]:
    # 早期上传记录按章节保存，生成阶段再按长度细分，兼顾知识完整性和接口响应速度。
    chunks = json.loads(book.chunks_json or "[]")
    sections: list[dict] = []
    for chunk in chunks:
        text = str(chunk.get("text", "")).strip()
        title = str(chunk.get("chapter", "") or "未命名小节").strip()
        if not text:
            continue
        for index, part in enumerate(_split_section(text), start=1):
            # 长小节拆分后保留共同标题，并用“续”标记顺序，方便大纲连续展示。
            clean_title = _clean_title(title)
            part_title = clean_title if index == 1 else f"{clean_title}（续 {index}）"
            sections.append({"chapter": part_title, "text": part})
    return sections

def _split_section(text: str) -> list[str]:
    # 优先在空行边界拆分，尽量不切断公式说明或同一段论证。
    if len(text) <= MAX_SECTION_CHARS:
        return [text]
    pieces = []
    paragraphs = [part.strip() for part in text.split("\n\n") if part.strip()]
    buf = ""
    for paragraph in paragraphs:
        # 只有缓冲区已有内容时才落盘，防止产生空片段。
        if len(buf) + len(paragraph) + 2 > MAX_SECTION_CHARS and buf:
            pieces.append(buf.strip())
            buf = paragraph
        else:
            buf = f"{buf}\n\n{paragraph}".strip()
    if buf:
        pieces.append(buf.strip())
    return pieces

def _delete_book_cards(db: Session, book_id: int) -> None:
    # 先删依赖卡片的聊天记录，再删卡片本身，满足外键约束并保持数据库整洁。
    card_ids = [row[0] for row in db.query(ConceptCard.id).filter(ConceptCard.book_id == book_id).all()]
    if card_ids:
        db.query(ChatMessage).filter(ChatMessage.card_id.in_(card_ids)).delete(synchronize_session=False)
        db.query(ConceptCard).filter(ConceptCard.book_id == book_id).delete(synchronize_session=False)

def _section_has_overview(db: Session, book_id: int, section_index: int) -> bool:
    # 导览卡按“书籍 + 小节 + 类型”判重，避免断点重试生成多张相同导览。
    return (
        db.query(ConceptCard)
        .filter(
            ConceptCard.book_id == book_id,
```



```
        ConceptCard.section_index == section_index,
        ConceptCard.card_type == "section_overview",
    )
    .first()
    is not None
)

def _build_card(book_id: int, section_index: int, chapter: str, section_text: str, card_data: dict) -> ConceptCard:
    # 模型输出属于不可信边界：类型使用白名单，缺少原文时回退到当前小节摘要。
    card_type = str(card_data.get("card_type") or "concept").strip()
    if card_type not in {"section_overview", "concept"}:
        card_type = "concept"
    source_text = str(card_data.get("source_text") or section_text[:1200]).strip()
    return ConceptCard(
        book_id=book_id,
        section_index=section_index,
        card_type=card_type,
        chapter=chapter,
        title=str(card_data.get("title") or "").strip(),
        source_text=source_text,
        one_sentence=str(card_data.get("one_sentence") or "").strip(),
        simple_explanation=str(card_data.get("simple_explanation") or "").strip(),
        fable=str(card_data.get("fable") or "").strip(),
        formula=str(card_data.get("formula") or "").strip(),
        formula_explanation=str(card_data.get("formula_explanation") or "").strip(),
        prerequisites=json.dumps(_as_str_list(card_data.get("prerequisites")), ensure_ascii=False),
        related_concepts=json.dumps(_as_str_list(card_data.get("related_concepts")), ensure_ascii=False),
        questions=json.dumps(_as_str_list(card_data.get("questions")), ensure_ascii=False),
    )

def _is_valid_card_title(title: str) -> bool:
    # 标题过滤是 Prompt 约束之外的第二道防线，专门拦截曾出现的前言噪声。
    if len(title) < 2 or len(title) > 100:
        return False
    lowered = title.lower().strip()
    blocked = {"department", "contents", "recommended books", "resources", "http://www", "this", "there"}
    return lowered not in blocked and not lowered.startswith("http")

def _is_probably_front_matter(text: str, chapter: str) -> bool:
    # 单个网址不足以判定为前置页；只有网址与目录/版权等信号共同出现才跳过。
    chapter_lowered = chapter.lower().strip()
    if any(signal in chapter_lowered for signal in ["contents", "recommended", "resources", "acknowledgement", "biblio...
        return True
    lowered = f"{chapter}\n{text}".lower()
    url_count = lowered.count("http://") + lowered.count("https://") + lowered.count("www.")
    front_signals = ["contents", "recommended books", "department", "copyright", "references", "bibliography"]
    signal_count = sum(1 for signal in front_signals if signal in lowered)
    return url_count >= 1 and signal_count >= 1

def _preview(text: str) -> str:
    collapsed = " ".join(text.split())
    return collapsed[:120]

def _clean_title(title: str) -> str:
    # 去除 Markdown/HTML 残留并折叠空白，保证标题可安全显示在大纲与卡片头部。
```

```
cleaned = re.sub(r"<[^>]+>", "", str(title or ""))
cleaned = re.sub(r"\s+", " ", cleaned).strip()
return cleaned or "未命名小节"

def _loads_list(value: str) -> list[str]:
    # 历史数据可能包含损坏 JSON，读取失败时返回空数组而不是让整页加载失败。
    try:
        data = json.loads(value or "[]")
        return _as_str_list(data)
    except json.JSONDecodeError:
        return []

def _as_str_list(value) -> list[str]:
    if not isinstance(value, list):
        return []
    return [str(item).strip() for item in value if str(item).strip()]

# ===== FILE: backend/services/concept_splitter.py =====

from __future__ import annotations
import re
from dataclasses import dataclass

HEADING_RE = re.compile(
    # 同时识别 Markdown 标题、中文章节目次以及英文 Chapter/Lecture 标题。
    r"^(#{1,6}\s+|第[一二三四五六七八九十百千万0-9]+[章节篇部].*(Chapter\s+\d+.*|Lecture\s+\d+.*))$",
    re.I,
)

@dataclass
class TextChunk:
    chapter: str
    text: str

def split_into_chunks(text: str, max_chars: int = 4200) -> list[dict]:
    # current_chapter 保存最近一次识别到的标题，后续普通段落都归入该章节。
    current_chapter = "未命名章节"
    current_parts: list[str] = []
    chunks: list[TextChunk] = []

    def flush():
        nonlocal current_parts
        # 遇到新标题或文本结束时，将当前章节按长度落为一个或多个稳定片段。
        joined = "\n\n".join(part for part in current_parts if part.strip()).strip()
        if joined:
            for piece in _split_long_text(joined, max_chars):
                chunks.append(TextChunk(chapter=current_chapter, text=piece))
            current_parts = []

    # 先按空行划分自然段；只有“单独占一段”的标题才改变章节，减少正文误判。
    blocks = re.split(r"\n\s*\n", text)

    for block in blocks:
        lines = [line.strip() for line in block.splitlines() if line.strip()]
        if len(lines) == 1 and HEADING_RE.match(lines[0]):
            flush()
            current_chapter = re.sub(r"^(#{1,6}\s*|", "", lines[0]).strip()
            continue
        current_parts.append(block.strip())

    flush()
```

```
# 没有识别到任何结构时仍按长度生成片段，保证普通 TXT 文件可用。
if not chunks and text.strip():
    chunks = [TextChunk(chapter=current_chapter, text=piece) for piece in _split_long_text(text, max_chars)]
return [{"chapter": chunk.chapter, "text": chunk.text} for chunk in chunks]
def _split_long_text(text: str, max_chars: int) -> list[str]:
    # 超长段落退化为按中英文句末标点切分，尽量保持完整句子进入同一请求。
    if len(text) <= max_chars:
        return [text]
    sentences = re.split(r"(?<=[。！？！?])\s+", text)
    pieces: list[str] = []
    buf = ""
    for sentence in sentences:
        if len(buf) + len(sentence) + 1 > max_chars and buf:
            pieces.append(buf.strip())
            buf = sentence
        else:
            buf = f"{buf} {sentence}".strip()
    if buf:
        pieces.append(buf.strip())
    return pieces
# ===== FILE: backend/services/deepseek_client.py =====
from __future__ import annotations
import json
from json import JSONDecodeError
from typing import Any
import httpx
from services.runtime_settings import get_deepseek_config
class DeepSeekNotConfiguredError(RuntimeError):
    pass
class DeepSeekClient:
    def __init__(self):
        # 每次业务请求创建客户端并读取当前运行时设置，使网页刚保存的配置立即生效。
        self.config = get_deepseek_config()
    async def generate_cards_from_section(
        self,
        section_text: str,
        section_title: str,
        target_count: int = 6,
        include_overview: bool = True,
    ) -> list[dict[str, Any]]:
        # mock 模式用于无密钥演示；真实模式必须明确配置 API Key，不能静默伪造结果。
        if self.config["mock"]:
            return self._mock_section_cards(section_text, section_title, target_count, include_overview)
        if not self.config["api_key"]:
            raise DeepSeekNotConfiguredError("请先在 DeepSeek 设置中填写 API Key。")
        # 导览卡规则根据数据库中是否已有导览动态切换，保证整节只有一个体系入口。
        overview_rule = (
            "第一张必须是 section_overview 类型的导览卡，介绍为什么需要这一节、它能做什么、会展开哪些下级概念，并用一...
            if include_overview
            else "不要生成导览卡，只生成 concept 类型的概念卡。"
```

```
    )
    prompt=f'""你是一位量子场论课程设计师和教材精读老师。请按“知识体系”而不是零散词条来处理下面这一节教材。

任务：

1. 阅读整节内容，输出 1 到 {target_count} 张学习卡片。
2. {overview_rule}
3. 导览卡 title 使用“导览：小节标题”的形式，card_type 为 "section_overview"。
4. 概念卡 card_type 为 "concept"，只选择对理解本节结构有用的核心概念、公式、推理方法或物理图像。
5. 不要提取目录、参考书、网址、作者机构、页眉页脚、普通英文单词。
6. 每张卡都要服务于本节知识体系：说明它在本节中的位置，以及它和其他概念的关系。
7. fable 要完整、具体、较长，建议 250-450 字。导览卡寓言概括整节；概念卡寓言解释该概念。
8. 所有公式必须使用标准 LaTeX。行内公式用  $...$ ，整行公式用 
$$...$$
。
9. formula 字段只写一个最核心的公式或空字符串。不要 Markdown 代码块，不要图片，不要 Unicode 近似公式。可以写裸公式，也...
10. formula_explanation 必须逐个解释公式符号和物理含义，也要用 LaTeX 表示符号。
11. 必须只输出 JSON 数组，不要 Markdown，不要额外说明。
```

JSON 对象格式：

```
{
  "card_type": "section_overview 或 concept",
  "title": "卡片标题",
  "source_text": "支撑卡片的原文片段",
  "one_sentence": "一句话说明",
  "simple_explanation": "系统性解释，说明本卡在本节中的作用",
  "fable": "完整寓言",
  "formula": "
$$S = \int d^4x \sqrt{-g} \mathcal{L}$$
",
  "formula_explanation": " $S$  表示作用量， $\mathcal{L}$  表示拉格朗日密度， $d^4x$  表示对四维时空积分。",
  "prerequisites": ["前置知识"],
  "related_concepts": ["关联概念"],
  "questions": ["推荐追问1", "推荐追问2", "推荐追问3"]
}
```

小节标题：

{section_title}

小节内容：

{section_text}""

```
    # 卡片属于结构化抽取任务，使用较低温度减少字段遗漏和 JSON 波动。
    content = await self._chat(prompt, temperature=0.25)

    try:
        data = _json_from_text(content)
    except JSONDecodeError as exc:
        raise RuntimeError(f"DeepSeek 返回内容不是合法 JSON：{content[:500]}") from exc

    # 即便 JSON 合法，也只接受“对象数组”；其他类型视为本轮没有可保存卡片。
    if not isinstance(data, list):
        return []

    return [item for item in data if isinstance(item, dict)]

async def answer_question(self, card: Any, question: str) -> str:
    # 追问提示包含卡片原文、已有解释和核心公式，让回答限定在当前知识点上下文。
    if self.config["mock"]:
        return (
            f'可以这样理解：你问的是“{question}”。围绕 {card.title}，先看它在当前小节中的位置，"
            f'再看它依赖的前提和关联概念。卡片中的核心提示是：{card.one_sentence}"
        )
    if not self.config["api_key"]:
```

```
        raise DeepSeekNotConfiguredError("请先在 DeepSeek 设置中填写 API Key。")

    prompt = f"""\n你是一位耐心、准确、擅长公式排版的量子场论老师。

当前卡片类型：{getattr(card, "card_type", "concept")}\n
当前概念/导览：\n
{card.title}\n
所属章节：\n
{card.chapter}\n
原文片段：\n
{card.source_text}\n
已有解释：\n
{card.simple_explanation}\n
核心公式：\n
{card.formula}\n
用户问题：\n
{question}\n
回答要求：\n
1. 用清晰分段回答，避免一整坨文字。
2. 所有公式必须使用标准 LaTeX。行内公式用  $...$ ，整行公式用 
$$...$$
。
3. 不要使用 Unicode 近似符号替代 LaTeX，例如不要只写  $\int$ 、 $\psi$ 、 $\phi$ ；应写  $\int$ 、 $\psi$ 、 $\phi$ 。
4. 解释每个公式符号的含义。
5. 回答要服务于“本节知识体系”，说明它和上下级概念的关系。"""

    # 追问允许略高温度以获得更自然的教学表达，但仍保持较低随机性。
    return await self._chat(prompt, temperature=0.35)

async def _chat(self, prompt: str, temperature: float = 0.35) -> str:
    # DeepSeek 使用与 OpenAI Chat Completions 兼容的请求格式，密钥只放在后端请求头。
    url = f"{self.config['api_base']}.rstrip('/')}/chat/completions"
    headers = {"Authorization": f"Bearer {self.config['api_key']}" }
    payload = {
        "model": self.config["model"],
        "messages": [{"role": "user", "content": prompt}],
        "temperature": temperature,
    }

    # 超时单独转换为可读错误，用户可以据此缩短小节或稍后重试。
    try:
        async with httpx.AsyncClient(timeout=35) as client:
            resp = await client.post(url, headers=headers, json=payload)
    except httpx.TimeoutException as exc:
        raise RuntimeError("DeepSeek API 请求超时，请稍后重试或减少单次生成内容。") from exc
    except httpx.HTTPError as exc:
        raise RuntimeError(f"DeepSeek API 网络连接失败：{exc}") from exc

    # 同时校验 HTTP 状态和响应 JSON 层级，避免把网关错误页当作模型答案。
    try:
        if resp.status_code >= 400:
            detail = resp.text[:500]
            raise RuntimeError(f"DeepSeek API 调用失败：HTTP {resp.status_code} {detail}")

        data = resp.json()
        return data["choices"][0]["message"]["content"]
    except (KeyError, IndexError, JSONDecodeError) as exc:
        raise RuntimeError(f"DeepSeek API 返回格式异常：{resp.text[:500]}") from exc

def _mock_section_cards(
```

```
self,
section_text: str,
section_title: str,
target_count: int,
include_overview: bool,
) -> list[dict[str, Any]]:
    # mock 数据保持与真实接口完全相同的字段结构，便于前端离线联调。
    cards: list[dict[str, Any]] = []
    if include_overview:
        cards.append(
            {
                "card_type": "section_overview",
                "title": f"导览：{section_title}",
                "source_text": section_text[:900],
                "one_sentence": f"这一节建立 {section_title} 的学习地图。",
                "simple_explanation": "真实模式下，DeepSeek 会先生成一张导览卡，说明本节为什么重要、能解决什么问题...",
                "fable": "一位学生进入一座新城市。旧办法是随机走进每条街，把看到的招牌都记下来；新办法是先登上钟楼...",
                "formula": "",
                "formula_explanation": "",
                "prerequisites": ["本节上下文", "章节目录"],
                "related_concepts": ["下级概念", "学习路径"],
                "questions": ["这一节解决什么问题？", "本节有哪些核心概念？", "这些概念如何连接？"],
            }
        )
    cards.append(
        {
            "card_type": "concept",
            "title": "示例概念",
            "source_text": section_text[:900],
            "one_sentence": "这是 mock 模式下的概念卡示例。",
            "simple_explanation": "关闭 mock 并填写 DeepSeek API Key 后，系统会按小节生成真实概念卡。",
            "fable": "老师把一节课拆成地图和路标：地图先说明方向，路标再指向关键转弯。概念卡就是那些路标，它们不会...",
            "formula": "$$S = \\int d^4x \\mathcal{L}$$",
            "formula_explanation": "$$S$ 是作用量，$\\mathcal{L}$ 是拉格朗日密度，$d^4x$ 表示四维时空体积元。",
            "prerequisites": ["DeepSeek API Key"],
            "related_concepts": ["导览卡", "概念卡"],
            "questions": ["如何生成真实卡片？", "为什么按小节生成？", "如何继续生成后续内容？"],
        }
    )
    return cards[:target_count]

def _json_from_text(text: str) -> Any:
    # 优先解析纯 JSON；若模型意外包裹 Markdown 代码块或说明文字，再截取首尾 JSON。
    cleaned = text.strip()
    if cleaned.startswith("`"):
        cleaned = cleaned.strip("`")
        cleaned = cleaned.removeprefix("json").strip()
    try:
        return json.loads(cleaned)
    except json.JSONDecodeError:
        # 同时兼容对象和数组起始符，并选择最早出现的位置作为候选 JSON 起点。
```

```
        start_candidates = [i for i in [cleaned.find("{"), cleaned.find("[")] if i >= 0]

    if not start_candidates:

        raise

    start = min(start_candidates)

    end = max(cleaned.rfind("}"), cleaned.rfind("]")

    return json.loads(cleaned[start : end + 1])

# ===== FILE: backend/services/file_parser.py =====

from pathlib import Path

import chardet

ALLOWED_EXTENSIONS = {".md", ".txt"}

def validate_book_file(filename: str) -> str:

    # 扩展名统一转小写，允许用户上传 BOOK.MD 等大小写不同的文件名。

    ext = Path(filename).suffix.lower()

    if ext not in ALLOWED_EXTENSIONS:

        raise ValueError("仅支持 .md 和 .txt 文件")

    return ext.lstrip(".")

def read_text_file(path: Path) -> str:

    # 先读取原始字节并通过 chardet 猜测编码，兼容 UTF-8、GBK 等常见电子书文本。

    data = path.read_bytes()

    detected = chardet.detect(data)

    encoding = detected.get("encoding") or "utf-8"

    try:

        text = data.decode(encoding)

    except UnicodeDecodeError:

        # 探测结果不可靠时回退 UTF-8 容错解码，避免单个坏字节导致整本书无法上传。

        text = data.decode("utf-8", errors="ignore")

    # 统一 Windows、Linux 和旧式 Mac 换行，后续章节切分不再依赖操作系统格式。

    return text.replace("\r\n", "\n").replace("\r", "\n").strip()

# ===== FILE: backend/services/kokoro_manager.py =====

from __future__ import annotations

import socket

import subprocess

import sys

import os

from pathlib import Path

from urllib.parse import urlparse

from services.runtime_settings import get_tts_config

_process: subprocess.Popen | None = None

def start_kokoro_if_enabled() -> None:

    # 仅当用户选择 Kokoro 时启动独立服务，其他 TTS 模式不占用模型内存和显存。

    config = get_tts_config()

    if not config["enabled"] or config["api_style"] != "kokoro":

        return

    parsed = urlparse(config["api_url"])

    host = parsed.hostname or "127.0.0.1"

    port = parsed.port or 9977

    # 自动启动只针对本机地址；远程 TTS 服务应由其服务器独立维护。

    if host not in {"127.0.0.1", "localhost"}:

        return

    # 端口已有服务时直接复用，避免后端重载产生重复模型进程。
```

```
if _port_open(host, port):
    return
backend_dir = Path(__file__).resolve().parents[1]
python_exe = backend_dir / ".venv" / "Scripts" / "python.exe"
if not python_exe.exists():
    python_exe = Path(sys.executable)
# 子进程隐藏运行，标准输出和错误写入日志，便于排查模型下载或 CUDA 问题。
log_dir = backend_dir / "logs"
log_dir.mkdir(exist_ok=True)
stdout = (log_dir / "kokoro.log").open("a", encoding="utf-8")
stderr = (log_dir / "kokoro.err.log").open("a", encoding="utf-8")
global _process
creationflags = getattr(subprocess, "CREATE_NO_WINDOW", 0)
creationflags |= getattr(subprocess, "CREATE_NEW_PROCESS_GROUP", 0)
creationflags |= getattr(subprocess, "DETACHED_PROCESS", 0)
# 强制 UTF-8 可避免中文语音文本在 Windows 子进程管道中被本地代码页破坏。
env = os.environ.copy()
env["PYTHONUTF8"] = "1"
env["PYTHONIOENCODING"] = "utf-8"
env["HF_HUB_DISABLE_SYMLINKS_WARNING"] = "1"
_process = subprocess.Popen(
    [
        str(python_exe),
        "-m",
        "uvicorn",
        "services.kokoro_service:app",
        "--host",
        "127.0.0.1",
        "--port",
        str(port),
    ],
    cwd=str(backend_dir),
    stdout=stdout,
    stderr=stderr,
    env=env,
    creationflags=creationflags,
)
def _port_open(host: str, port: int) -> bool:
    # 使用短连接做轻量存活探测，启动流程不会因可达端口长时间阻塞。
    try:
        with socket.create_connection((host, port), timeout=0.5):
            return True
    except OSError:
        return False
# ===== FILE: backend/services/kokoro_service.py =====
from __future__ import annotations
import re
from io import BytesIO
from threading import Lock
import numpy as np
```



```
import soundfile as sf

from fastapi import FastAPI, HTTPException, Response

from pydantic import BaseModel

app = FastAPI(title="Local Kokoro TTS Service")

_pipelines: dict[tuple[str, str], object] = {}

# 模型管线按 “ 语言 + 设备 ” 缓存；锁用于阻止多个请求同时重复加载同一份模型。

_pipeline_lock = Lock()

_load_error = ""

SAMPLE_RATE = 24000

CHINESE_RE = re.compile(r"[\u3400-\u9fff]")

SPEECH_SYMBOL_RE = re.compile(r"“ ” „ “ ” ‘ ’ , ‘ ’ ’ () \[\] \{\} \{ } <> 《》 ◇ | | *_#'^~/@\\+=|- — — … · • ]")

class TTSRequest(BaseModel):

    text: str

    voice: str | None = "zf_xiaoxiao"

    english_voice: str | None = "af_heart"

    model: str | None = "kokoro-82m"

    lang_code: str | None = "auto"

    device: str | None = "auto"

    speed: float | None = 1.0

    stream: bool | None = False

@app.get("/health")

def health():

    auto_device = _resolve_device("auto")

    return {

        "status": "ok" if not _load_error else "error",

        "engine": "kokoro",

        "loaded_pipelines": [{ "lang_code": lang, "device": device } for lang, device in sorted(_pipelines.keys())],

        "default_lang_code": "auto",

        "supported_lang_codes": ["auto", "z", "zh", "a", "b"],

        "default_device": "auto",

        "resolved_auto_device": auto_device,

        "default_chinese_voice": "zf_xiaoxiao",

        "default_english_voice": "af_heart",

        "chinese_device": auto_device,

        "english_device": auto_device,

        "torch": _torch_status(),

        "error": _load_error,

    }

@app.post("/tts")

async def tts(payload: TTSRequest):

    # 输入先清洗特殊符号，再按句号拆句，降低长文本一次推理造成的停顿和显存峰值。

    sentences = _split_sentences(_sanitize_for_speech(payload.text))

    if not sentences:

        raise HTTPException(status_code=400, detail="朗读文本不能为空。")

    # auto 优先使用 CUDA；每句话再按中英文切段，为不同语言选择对应音色。

    requested_device = _resolve_device(payload.device)

    pieces: list[np.ndarray] = []

    for sentence in sentences:

        for segment_text, lang_code, voice in _segments_for_text(sentence, payload):

            pipeline = _get_pipeline(lang_code, requested_device)
```

```
try:
    # 已由本层完成分句，因此禁用 Kokoro 内部分割，避免产生双重停顿。
    generator = pipeline(
        segment_text,
        voice=voice,
        speed=payload.speed or 1.0,
        split_pattern=r"$^",
    )
    pieces.extend(_to_numpy(result.audio) for result in generator if getattr(result, "audio", None) is not...)
except Exception as exc:
    raise HTTPException(status_code=500, detail=f"Kokoro 推理失败：{type(exc).__name__}: {exc}") from exc

if not pieces:
    raise HTTPException(status_code=500, detail="Kokoro 没有返回音频。")
# 各句音频按顺序无缝拼接成标准 WAV，浏览器可直接播放并进行磁盘缓存。
audio = np.concatenate(pieces).astype(np.float32)
buffer = BytesIO()
sf.write(buffer, audio, SAMPLE_RATE, format="WAV")
return Response(content=buffer.getvalue(), media_type="audio/wav")

@app.post("/preload")
async def preload(payload: dict | None = None):
    # auto 语言会同时预热中文和英文管线，首次遇到混合文本时无需再次加载。
    payload = payload or {}
    requested_device = _resolve_device(payload.get("device", "auto"))
    lang_code = _normalize_lang_code(payload.get("lang_code", "auto"))
    lang_codes = ["z", "a"] if lang_code == "auto" else [lang_code]
    for code in lang_codes:
        _get_pipeline(code, requested_device)
    return {
        "status": "ok",
        "device": requested_device,
        "lang_code": lang_code,
        "loaded_pipelines": [{"lang_code": lang, "device": device} for lang, device in sorted(_pipelines.keys())],
    }

def _split_sentences(text: str) -> list[str]:
    # 为每段补回句号，让模型保留自然的句末语调。
    return [f"{item.strip()}。" for item in text.split("。") if item.strip()]

def _segments_for_text(text: str, payload: TTSRequest) -> list[tuple[str, str, str]]:
    # 指定语言时整句使用固定管线；auto 模式才逐字符识别中英文边界。
    lang_code = _normalize_lang_code(payload.lang_code)
    if lang_code != "auto":
        return [(text, lang_code, _voice_for_lang(lang_code, payload))]
    segments: list[tuple[str, str, str]] = []
    current_lang = "z" if _has_chinese(text[:1]) else "a"
    current = ""
    # 空格不触发语言切换，避免英文单词之间被拆成大量短音频。
    for char in text:
        char_lang = "z" if _has_chinese(char) else "a"
        if current and char_lang != current_lang and not char.isspace():
            if current.strip():
                segments.append((current.strip(), current_lang, _voice_for_lang(current_lang, payload)))
```

```

        current = char

        current_lang = char_lang

    else:

        current += char

if current.strip():

    segments.append((current.strip(), current_lang, _voice_for_lang(current_lang, payload)))

return segments

def _normalize_lang_code(lang_code: str | None) -> str:

    value = (lang_code or "auto").strip().lower()

    aliases = {"zh": "z", "cn": "z", "mandarin": "z", "en": "a", "us": "a", "american": "a", "uk": "b", "british": "b"...

    value = aliases.get(value, value)

    if value not in {"auto", "z", "a", "b"}:

        return "auto"

    return value

def _voice_for_lang(lang_code: str, payload: TTSTRequest) -> str:

    if lang_code == "z":

        return payload.voice or "zf_xiaoxiao"

    return payload.english_voice or "af_heart"

def _has_chinese(text: str) -> bool:

    return bool(CHINESE_RE.search(text))

def _sanitize_for_speech(text: str) -> str:

    # 冒号、分号等统一为可控停顿，只保留中英文、数字、中文逗号和句号。

    # 此转换仅影响送入语音模型的副本，不修改页面显示的原始卡片内容。

    cleaned = str(text or "")

    cleaned = re.sub(r"[ ,\n ]", "", cleaned)

    cleaned = re.sub(r"[.!?;! ? ; ; : : \n\r]+", "。", cleaned)

    cleaned = SPEECH_SYMBOL_RE.sub("", cleaned)

    cleaned = re.sub(r"[^\w\u3400-\u9fff, 。\s]", "", cleaned, flags=re.UNICODE)

    cleaned = re.sub(r"\s*", "", cleaned)

    cleaned = re.sub(r"\s*。", "", cleaned)

    cleaned = re.sub(r" +", "", cleaned)

    cleaned = re.sub(r"。 +", "", cleaned)

    cleaned = re.sub(r"\s+", "", cleaned)

    return cleaned.strip()

def _resolve_device(device: str | None) -> str:

    # 用户强制 CUDA 但环境不可用时安全回退 CPU，避免整个 TTS 接口直接崩溃。

    value = (device or "auto").strip().lower()

    if value in {"cpu", "cuda"}:

        if value == "cuda" and not _cuda_available():

            return "cpu"

        return value

    return "cuda" if _cuda_available() else "cpu"

def _cuda_available() -> bool:

    try:

        import torch

        return bool(torch.cuda.is_available())

    except Exception:

        return False

def _torch_status() -> dict:

    try:
```



```
# worker 使用 “ 一行一个 JSON ” 的协议与主进程通信， 模型可常驻并跨请求复用。
for line in sys.stdin:
    try:
        payload = json.loads(line)
        op = payload.get("op")
        if op == "health":
            result = health()
        elif op == "preload":
            result = preload(payload)
        elif op == "tts":
            result = tts(payload)
        else:
            result = {"ok": False, "error": f"未知操作：{op}"}
    except Exception as exc:
        result = {"ok": False, "error": f"{type(exc).__name__}: {exc}"}
# stdout 只输出协议响应；模型自身日志被重定向到 stderr，避免污染 JSON。
sys.stdout.write(json.dumps(result, ensure_ascii=False) + "\n")
sys.stdout.flush()

def health() -> dict:
    device = _resolve_device("auto")
    return {
        "ok": True,
        "engine": "kokoro-worker",
        "device": device,
        "loaded_pipelines": [{"lang_code": lang, "device": item_device} for lang, item_device in sorted(_pipelines.key...
        "torch": _torch_status(),
    }

def preload(payload: dict) -> dict:
    # 预加载只构建管线，不产生音频；auto 模式同时准备中英文模型。
    requested_device = _resolve_device(payload.get("device", "auto"))
    lang_code = _normalize_lang_code(payload.get("lang_code", "auto"))
    lang_codes = ["z", "a"] if lang_code == "auto" else [lang_code]
    for code in lang_codes:
        _get_pipeline(code, requested_device)
    return {
        "ok": True,
        "device": requested_device,
        "lang_code": lang_code,
        "loaded_pipelines": [{"lang_code": lang, "device": item_device} for lang, item_device in sorted(_pipelines.key...
    }

def tts(payload: dict) -> dict:
    # 长文本先按句号拆分，再按语言分段，逐段推理后拼接成一个 WAV 响应。
    sentences = _split_sentences(_sanitize_for_speech(payload.get("text", "")))
    if not sentences:
        return {"ok": False, "error": "朗读文本不能为空。"}
    requested_device = _resolve_device(payload.get("device", "auto"))
    request = TTSLike(payload)
    pieces: list[np.ndarray] = []
    for sentence in sentences:
        for segment_text, lang_code, voice in _segments_for_text(sentence, request):
```

```

    pipeline = _get_pipeline(lang_code, requested_device)

    # Kokoro 可能向 stdout 打印下载进度，必须重定向以保护进程通信协议。
    with contextlib.redirect_stdout(sys.stderr):
        generator = pipeline(
            segment_text,
            voice=voice,
            speed=request.speed,
            split_pattern=r"$^",
        )

        pieces.extend(_to_numpy(result.audio) for result in generator if getattr(result, "audio", None) is not...)

if not pieces:
    return {"ok": False, "error": "Kokoro 没有返回音频。"}

# 音频以 Base64 放入 JSON 返回；主进程解码后再写入缓存或交给浏览器。
audio = np.concatenate(pieces).astype(np.float32)

buffer = BytesIO()
sf.write(buffer, audio, SAMPLE_RATE, format="WAV")

return {
    "ok": True,
    "content_type": "audio/wav",
    "audio": base64.b64encode(buffer.getvalue()).decode("ascii"),
}

class TTSLike:
    def __init__(self, payload: dict):
        self.voice = payload.get("voice") or "zf_xiaoxiao"
        self.english_voice = payload.get("english_voice") or "af_heart"
        self.lang_code = payload.get("lang_code") or "auto"
        self.speed = float(payload.get("speed") or 1.0)

def _split_sentences(text: str) -> list[str]:
    return [f"{item.strip()}。" for item in text.split("。") if item.strip()]

def _segments_for_text(text: str, payload: TTSLike) -> list[tuple[str, str, str]]:
    # 中文使用 z 管线和中文音色，英文使用 a/b 管线和独立英文音色。
    lang_code = _normalize_lang_code(payload.lang_code)

    if lang_code != "auto":
        return [(text, lang_code, _voice_for_lang(lang_code, payload))]

    segments: list[tuple[str, str, str]] = []
    current_lang = "z" if _has_chinese(text[:1]) else "a"
    current = ""

    for char in text:
        char_lang = "z" if _has_chinese(char) else "a"

        if current and char_lang != current_lang and not char.isspace():
            if current.strip():
                segments.append((current.strip(), current_lang, _voice_for_lang(current_lang, payload)))

            current = char
            current_lang = char_lang
        else:
            current += char

    if current.strip():
        segments.append((current.strip(), current_lang, _voice_for_lang(current_lang, payload)))

    return segments

def _normalize_lang_code(lang_code: str | None) -> str:
```

```
    "voice": config["voice"],
    "english_voice": config.get("english_voice", "af_heart"),
    "model": config["model"],
    "lang_code": config.get("lang_code", "z"),
    "device": config.get("device", "auto"),
    "speed": resolved_speed,
    "stream": False,
}

def _clamp_speed(value: float | str | None) -> float:
    # 合法范围统一为 0.2 至 2.0；解析失败时回到产品默认值 0.8。
    try:
        parsed = float(value)
    except (TypeError, ValueError):
        return 0.8
    return min(2.0, max(0.2, parsed))

async def _post_tts(config: dict, payload: dict, path: str | None = None) -> httpx.Response:
    url = _service_url(config, path)
    # 本机 TTS 不继承系统代理，避免 127.0.0.1 请求被代理软件转发后失效。
    async with httpx.AsyncClient(timeout=90, trust_env=False) as client:
        try:
            response = await client.post(url, json=payload)
            if response.status_code >= 400:
                raise RuntimeError(f"TTS 服务返回错误：HTTP {response.status_code} {response.text[:300]}")
            return response
        except httpx.TimeoutException as exc:
            raise RuntimeError("TTS 生成超时，请稍后重试或缩短朗读文本。") from exc
        except httpx.HTTPError as exc:
            raise RuntimeError(f"TTS 服务连接失败：{exc}") from exc

def _service_url(config: dict, path: str | None) -> str:
    if not path:
        return config["api_url"]
    base = config["api_url"].rsplit("/", 1)[0]
    return f"{base}/{path}"

def _cache_key(payload: dict, config: dict) -> str:
    # JSON 键排序后计算 SHA-256，得到稳定且不暴露原文内容的缓存文件名。
    material = {"style": config.get("api_style"), "url": config.get("api_url"), "payload": payload}
    raw = json.dumps(material, ensure_ascii=False, sort_keys=True).encode("utf-8")
    return hashlib.sha256(raw).hexdigest()

def _write_cache(cache_path: Path, audio: bytes, content_type: str) -> None:
    # 只缓存非空音频响应，错误页或空结果不会污染后续朗读。
    if not audio or not content_type.startswith("audio/"):
        return
    CACHE_DIR.mkdir(exist_ok=True)
    cache_path.write_bytes(audio)

// ===== FILE: frontend/src/api/client.js =====
const API_BASE = import.meta.env.VITE_API_BASE || "http://localhost:8000";

async function request(path, options = {}) {
    // JSON 请求统一经过此入口，使错误信息始终优先使用后端 detail 字段。
    // API_BASE 可在构建时覆盖，开发环境默认连接本机 FastAPI 服务。
    const response = await fetch(`${API_BASE}${path}`, options);
```

```
const text = await response.text();

const data = text ? JSON.parse(text) : null;

if (!response.ok) {
  throw new Error(data?.detail || "请求失败");
}

return data;
}

async function requestBlob(path, options = {}) {
  // 语音接口返回二进制 Blob；失败响应仍按 JSON/文本解析，不能直接当音频播放。
  const response = await fetch(`${API_BASE}${path}`, options);

  if (!response.ok) {
    const text = await response.text();
    let detail = "请求失败";

    try {
      detail = JSON.parse(text)?.detail || detail;
    } catch {}

    detail = text || detail;
  }

  throw new Error(detail);
}

return response.blob();
}

export const api = {
  // 页面组件只调用语义化方法，不在各处重复拼接路径、请求头和序列化规则。
  listBooks: () => request("/api/books"),
  getBook: (bookId) => request(`/api/books/${bookId}`),
  uploadBook: (file) => {
    // 浏览器自动为 FormData 生成带 boundary 的 Content-Type，不能手动覆盖请求头。
    const body = new FormData();
    body.append("file", file);
    return request("/api/upload", { method: "POST", body });
  },
  generateCards: (bookId, { force = false } = {}) =>
    request(`/api/books/${bookId}/generate-cards?force=${force ? "true" : "false"}`, { method: "POST" }),
  getOutline: (bookId) => request(`/api/books/${bookId}/outline`),
  listCards: (bookId) => request(`/api/books/${bookId}/cards`),
  toggleFavorite: (cardId) => request(`/api/cards/${cardId}/favorite`, { method: "POST" }),
  listMessages: (cardId) => request(`/api/cards/${cardId}/chat`),
  ask: (cardId, question) =>
    request(`/api/cards/${cardId}/chat`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ question }),
    }),
  getDeepSeekSettings: () => request("/api/settings/deepseek"),
  updateDeepSeekSettings: (payload) =>
    request("/api/settings/deepseek", {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),
    })
}
```



```
    }),
    getTTSSettings: () => request("/api/settings/tts"),
    updateTTSSettings: (payload) =>
      request("/api/settings/tts", {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload),
      }),
    preloadTTS: () =>
      request("/api/tts/preload", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
      }),
    synthesizeSpeech: (text, options = {}) =>
      requestBlob("/api/tts/speech", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text, ...options }),
      }),
  };

// ===== FILE: frontend/src/App.jsx =====
import { useEffect, useState } from "react";
import { ArrowLeft, BookOpen, KeyRound, Library } from "lucide-react";
import { api } from "../api/client";
import UploadPage from "../pages/UploadPage";
import BookPage from "../pages/BookPage";
import CardReaderPage from "../pages/CardReaderPage";
import DeepSeekSettings from "../components/DeepSeekSettings";
export default function App() {
  // MVP 不依赖路由库，顶层 route 状态在上传、书籍详情和沉浸阅读三种视图间切换。
  // 书籍数据集中保存在 App，上传完成后刷新即可同步顶部数量和书库列表。
  const [route, setRoute] = useState({ name: "upload" });
  const [books, setBooks] = useState([]);
  const [settingsOpen, setSettingsOpen] = useState(false);
  const refreshBooks = async () => {
    // 后端不可达时保留可渲染的空列表，避免整个应用因首页请求失败而白屏。
    try {
      setBooks(await api.listBooks());
    } catch {
      setBooks([]);
    }
  };

  useEffect(() => {
    refreshBooks();
  }, []);

  return (
    <main className="min-h-screen">
      <header className="fixed left-0 right-0 top-0 z-20 border-b border-stone-200 bg-[#f7f5ef]/95 backdrop-blur">
        <div className="mx-auto flex h-14 max-w-6xl items-center justify-between px-4">
          <button
```

```

        className="flex items-center gap-2 text-sm font-semibold text-stone-900"
        onClick={() => setRoute({ name: "upload" })}}
    >
    <BookOpen size={18} />
    AI 书籍概念学习
</button>
<div className="flex items-center gap-2">
    {route.name !== "upload" && (
        <button
            className="inline-flex h-9 items-center gap-2 rounded-md border border-stone-300 px-3 text-sm hover:bg...
            onClick={() => setRoute({ name: "upload" })}}
        >
        <ArrowLeft size={16} />
        返回
    </button>
    )}
    <button
        className="inline-flex h-9 items-center gap-2 rounded-md border border-stone-300 px-3 text-sm hover:bg-w...
        onClick={() => setSettingsOpen(true)}
        title="DeepSeek 与语音设置"
    >
    <KeyRound size={16} />
    设置
    </button>
    <button
        className="inline-flex h-9 items-center gap-2 rounded-md bg-stone-900 px-3 text-sm text-white"
        onClick={() => setRoute({ name: "upload" })}}
        title="书库"
    >
    <Library size={16} />
    {books.length}
    </button>
</div>
</div>
</header>
<div className="pt-14">
    {/* 三个页面共享固定顶部栏，但只挂载当前页面，避免隐藏页面继续发起请求。 */}
    {route.name === "upload" && (
        <UploadPage
            books={books}
            onUploaded={async (bookId) => {
                await refreshBooks();
                setRoute({ name: "book", bookId });
            }}
            onOpenBook={({bookId) => setRoute({ name: "book", bookId })}}
        />
    )}
    {route.name === "book" && (
        <BookPage
            bookId={route.bookId}

```

```
      onRead={bookId => setRoute({ name: "reader", bookId })}}

    />
  )}

  {route.name === "reader" && <CardReaderPage bookId={route.bookId} />}

</div>

<DeepSeekSettings open={settingsOpen} onClose={() => setSettingsOpen(false)} />

</main>

);
}

// ===== FILE: frontend/src/components/ChatPanel.jsx =====
import { Send } from "lucide-react";
import { useEffect, useState } from "react";
import { api } from "../api/client";
import RichText from "../RichText";
export default function ChatPanel({ card }) {
  const [messages, setMessages] = useState([]);
  const [question, setQuestion] = useState("");
  const [busy, setBusy] = useState(false);
  useEffect(() => {
    // 切换活动卡片时重新加载该卡片的独立历史，避免不同概念的对话混在一起。
    if (!card) return;
    api.listMessages(card.id).then(setMessages).catch(() => setMessages([]));
  }, [card?.id]);
  const ask = async (preset) => {
    const content = (preset || question).trim();
    if (!content || !card) return;
    setBusy(true);
    setQuestion("");
    // 先乐观显示用户问题，减少网络等待造成的“点击无反应”感。
    setMessages((items) => [...items, { role: "user", content }]);
    try {
      const data = await api.ask(card.id, content);
      // 模型回答交给 RichText 解析，使段落、列表和 LaTeX 公式保持可读排版。
      setMessages((items) => [...items, { role: "assistant", content: data.answer }]);
    } finally {
      setBusy(false);
    }
  };
  return (
    <aside className="hidden h-[calc(100vh-56px)] border-l border-stone-200 bg-white lg:flex lg:flex-col">
      <div className="border-b border-stone-100 p-4">
        <h2 className="font-semibold text-stone-950">追问当前卡片</h2>
        <p className="mt-1 truncate text-sm text-stone-500">{card?.title || "未选择卡片"}</p>
      </div>
      <div className="space-y-2 border-b border-stone-100 p-3">
        {/* 推荐问题与自由输入共用 ask，保证上下文绑定和保存逻辑完全一致。 */}
        {card?.questions?.map((item) => (
          <button key={item} className="block w-full rounded-md bg-stone-100 px-3 py-2 text-left text-sm leading-5 hov...
            {item}
          </button>
        ))}
      </div>
    </aside>
  );
}
```

```
    )})
  </div>

  <div className="flex-1 space-y-4 overflow-auto p-4">
    {messages.map((message, index) => (
      <div key={index} className={message.role === "user" ? "text-right" : "text-left"}>
        <div className={`inline-block max-w-[94%] rounded-md px-3 py-2 text-left ${message.role === "user" ? "bg-s...
          {message.role === "user" ? (
            <p className="text-sm leading-6">{message.content}</p>
          ) : (
            <RichText text={message.content} />
          )}
        </div>
      </div>
    ))}
  </div>

  <form
    className="flex gap-2 border-t border-stone-100 p-3"
    onSubmit={(event) => {
      event.preventDefault();
      ask();
    }}
  >
    <textarea
      className="min-h-10 min-w-0 flex-1 resize-none rounded-md border border-stone-300 px-3 py-2 text-sm outline-...
      value={question}
      rows={1}
      onChange={(event) => setQuestion(event.target.value)}
      placeholder="继续追问..."
    />
    <button className="inline-flex h-10 w-10 items-center justify-center rounded-md bg-stone-900 text-white disabl...
      <Send size={17} />
    </button>
  </form>
</aside>

);
}

// ===== FILE: frontend/src/components/ConceptCard.jsx =====
import { motion } from "framer-motion";
import { Check, Clipboard, Heart, Layers, MessageSquare, Quote, Star } from "lucide-react";
import { useState } from "react";
import { api } from "../api/client";
import VoiceButton from "./VoiceButton";
import RichText from "./RichText";

export default function ConceptCard({ card, index, total, onFavorite }) {
  // 原文、知识关系和复制反馈都是卡片自身的临时界面状态，切换卡片后互不影响。
  const [copied, setCopied] = useState(false);
  const [showSource, setShowSource] = useState(false);
  const [deep, setDeep] = useState(false);

  // 复制内容按学习顺序组织，保留换行方便粘贴到笔记软件继续整理。
  const text = `${card.title}\n${card.one_sentence}\n\n${card.simple_explanation}\n\n${card.fable}\n\n${card.formula}` ...
```

```
const isOverview = card.card_type === "section_overview";

const copy = async () => {
  await navigator.clipboard.writeText(text);
  setCopied(true);
  setTimeout(() => setCopied(false), 1200);
};

const favorite = async () => {
  // 以后端返回值为准更新父级列表，避免本地乐观切换与数据库状态不一致。
  const data = await api.toggleFavorite(card.id);
  onFavorite(card.id, data.is_favorite);
};

return (
  <motion.article
    initial={{ opacity: 0.6, y: 20 }}
    whileInView={{ opacity: 1, y: 0 }}
    viewport={{ amount: 0.7 }}
    className="flex max-h-full w-full max-w-3xl flex-col overflow-hidden rounded-md border border-stone-200 bg-white...
  >
    <div className="flex items-center justify-between border-b border-stone-100 px-5 py-3">
      <span className="text-sm text-stone-500">
        {index + 1} / {total} · {isOverview ? "小节导览": "概念"} · {card.chapter}
      </span>
      <div className="flex items-center gap-2">
        <VoiceButton card={card} text={card.simple_explanation} />
        <IconButton onClick={copy} title="复制">{copied ? <Check size={18} /> : <Clipboard size={18} />}</IconButton...
        <IconButton onClick={favorite} title="收藏">
          <Heart size={18} className={card.is_favorite ? "fill-red-500 text-red-500": ""} />
        </IconButton>
      </div>
    </div>
    <div>
      <div className="overflow-y-auto p-5">
        <h1 className="text-2xl font-bold text-stone-950">{card.title}</h1>
        <p className={mt-3 rounded-md p-3 leading-7 ${isOverview ? "bg-amber-50 text-amber-950": "bg-[#ecf3ff] text-...
          {card.one_sentence}
        </p>
        <Section title={isOverview ? "体系导览": "通俗解释"} icon={isOverview ? <Layers size={16} /> : <Star size={16...
          <RichText text={card.simple_explanation} />
        </Section>
        {/* 寓言按钮显式关闭上下文拼接，只朗读当前寓言，不重复标题与一句话解释。 */}
        <Section
          title="寓言故事"
          icon={<Quote size={16} />}
          action={<VoiceButton text={card.fable} label="读寓言" compact={false} includeContext={false} />}
        >
          <RichText text={card.fable} />
        </Section>
        <Section title="核心公式" icon={<MessageSquare size={16} />}>
          <FormulaText formula={card.formula} />
          {card.formula_explanation && <div className="mt-2"><RichText text={card.formula_explanation} /></div>}
        </Section>
```

```
<button className="mt-4 text-sm font-semibold text-stone-900 underline" onClick={() => setDeep(!deep)}>
  {deep ? "收起知识关系" : "展开知识关系"}
</button>

{deep && (
  <div className="mt-3 grid gap-3 sm:grid-cols-2">
    <PillList title="前置知识" items={card.prerequisites} />
    <PillList title="关联概念" items={card.related_concepts} />
  </div>
)}

<button className="mt-4 text-sm font-semibold text-stone-900 underline" onClick={() => setShowSource(!showSour...
  {showSource ? "隐藏原文片段" : "查看原文片段"}
</button>

{showSource && <p className="mt-3 max-h-40 overflow-auto rounded-md border border-stone-200 bg-stone-50 p-3 te...
</div>
</motion.article>
);
}

function FormulaText({ formula }) {
  const normalized = normalizeFormula(formula);
  if (!normalized) return <RichText text="无明确公式" />;
  return <RichText text={normalized} />;
}

function normalizeFormula(value) {
  // DeepSeek 可能返回裸公式或四种常见边界，统一包装后交由 RichText/KaTeX 渲染。
  // 已经带边界的公式保持原样，防止重复包裹产生无法解析的美元符号。
  const raw = String(value || "").trim();
  if (!raw) return "";
  if (/^\$\$[\s\S]*\$\$/ .test(raw)) return raw;
  if (/^\[\[\[\[\[\s\S]*\]\]\]\]\$/ .test(raw)) return raw.replace(/^\[\[\[\[\[/, "$$").replace(/\]\]\]\]/, "$$");
  if (/^\$[\^$\][\s\S]*\$/ .test(raw)) return raw;
  if (/^\[\[\[\([\s\S]*\)\]\]\]\$/ .test(raw)) return raw.replace(/^\[\[\[\[/, "$$").replace(/\)\]\]\]/, "$$");
  return `$$$${raw}$$$`;
}

function IconButton({ children, onClick, title }) {
  return (
    <button className="inline-flex h-9 w-9 items-center justify-center rounded-md border border-stone-200 hover:bg-sto...
      {children}
    </button>
  );
}

function Section({ title, icon, action, children }) {
  return (
    <section className="mt-5">
      <div className="mb-2 flex items-center justify-between gap-2">
        <h2 className="flex items-center gap-2 text-sm font-semibold text-stone-500">{icon}{title}</h2>
        {action}
      </div>
      <div>{children}</div>
    </section>
  );
}
```

```
}

function PillList({ title, items }) {

  // 前置知识与关联概念采用同一小组件，保证空格、换行和移动端折行规则一致。

  return (

    <div className="rounded-md border border-stone-200 p-3">

      <h3 className="mb-2 text-sm font-semibold text-stone-500">{title}</h3>

      <div className="flex flex-wrap gap-2">

        {items.map((item) => <span key={item} className="rounded-md bg-stone-100 px-2 py-1 text-sm">{item}</span>)}

      </div>

    </div>

  );

}

// ===== FILE: frontend/src/components/DeepSeekSettings.jsx =====

import { KeyRound, Loader2, Save, Volume2, X } from "lucide-react";

import { useEffect, useRef, useState } from "react";

import { api } from "../api/client";

const SPEED_STORAGE_KEY = "book-concept-app.tts.speed";

// 音色使用受支持的固定选项，避免手工输入不存在的 voice 导致模型请求失败。

const CHINESE_VOICES = [

  { value: "zf_xiaoxiao", label: "zf_xiaoxiao 女声，普通话" },

  { value: "zf_001", label: "zf_001 女声，普通话" },

  { value: "zm_yunxi", label: "zm_yunxi 男声，普通话" },

  { value: "zm_009", label: "zm_009 男声，普通话" },

];

const ENGLISH_VOICES = [

  { value: "af_heart", label: "af_heart 女声，美式英语" },

  { value: "af_bella", label: "af_bella 女声，美式英语" },

  { value: "am_adam", label: "am_adam 男声，美式英语" },

  { value: "bf_emma", label: "bf_emma 女声，英式英语" },

  { value: "bm_george", label: "bm_george 男声，英式英语" },

];

const SHERPA_CHINESE_VOICES = [

  { value: "zh_CN-xiao_ya-medium", label: "xiao_ya 女声，普通话" },

  { value: "zh_CN-chaowen-medium", label: "chaowen 男声，普通话" },

];

const SHERPA_ENGLISH_VOICES = [

  { value: "en_US-lessac-medium", label: "lessac 女声，美式英语" },

];

export default function DeepSeekSettings({ open, onClose }) {

  // API Key 输入框只保存本次新值；后端返回的 masked 仅用于提示已有配置。

  const [apiKey, setApiKey] = useState("");

  const [apiBase, setApiBase] = useState("https://api.deepseek.com");

  const [model, setModel] = useState("deepseek-chat");

  const [mock, setMock] = useState(false);

  const [masked, setMasked] = useState("");

  const [ttsEnabled, setTtsEnabled] = useState(true);

  const [ttsUrl, setTtsUrl] = useState("http://127.0.0.1:9977/tts");

  const [ttsStyle, setTtsStyle] = useState("kokoro");

  const [ttsVoice, setTtsVoice] = useState("zf_xiaoxiao");

  const [ttsEnglishVoice, setTtsEnglishVoice] = useState("af_heart");

}
```

```
const [ttsModel, setTtsModel] = useState("kokoro-82m");
const [ttsLangCode, setTtsLangCode] = useState("z");
const [ttsDevice, setTtsDevice] = useState("auto");
// 语速先从 localStorage 恢复，使设置窗打开前的朗读也使用上次选择。
const [ttsSpeed, setTtsSpeed] = useState(() => readStoredSpeed(0.8));
const speedSaveTimer = useRef(null);
const [busy, setBusy] = useState(false);
const [preloading, setPreloading] = useState(false);
const [message, setMessage] = useState("");
useEffect(() => {
  if (!open) return;
  // 设置窗打开时并行读取两组配置，减少串行请求造成的界面等待。
  Promise.all([api.getDeepSeekSettings(), api.getTTSSettings()]).then(([deepseek, tts]) => {
    setApiBase(deepseek.api_base);
    setModel(deepseek.model);
    setMock(deepseek.mock);
    setMasked(deepseek.api_key_masked);
    setTtsEnabled(tts.enabled);
    setTtsUrl(tts.api_url);
    setTtsStyle(tts.api_style);
    setTtsVoice(tts.voice || "zf_xiaoxiao");
    setTtsEnglishVoice(tts.english_voice || "af_heart");
    setTtsModel(tts.model || "kokoro-82m");
    setTtsLangCode(tts.lang_code || "z");
    setTtsDevice(tts.device || "auto");
    // 本地值优先用于即时体验，服务端值作为首次使用或清理缓存后的回退。
    const speed = readStoredSpeed(tts.speed ?? 0.8);
    setTtsSpeed(speed);
    storeSpeed(speed);
  });
}, [open]);
if (!open) return null;
const updateSpeed = (value) => {
  // 拖动滑块时立即更新本地值，并用防抖减少连续 PUT 请求。
  const speed = clampSpeed(value);
  setTtsSpeed(speed);
  storeSpeed(speed);
  window.clearTimeout(speedSaveTimer.current);
  speedSaveTimer.current = window.setTimeout(() => {
    api.updateTTSSettings({ speed }).catch((err) => setMessage(err.message));
  }, 250);
};
const selectTtsStyle = (style) => {
  // 切换引擎时同步切换其有效默认模型、语言代码、音色和设备组合。
  // 这些默认值只更新表单，用户点击保存或预加载后才写入后端。
  setTtsStyle(style);
  if (style === "sherpa") {
    setTtsVoice("zh_CN-xiao_ya-medium");
    setTtsEnglishVoice("en_US-lessac-medium");
    setTtsModel("sherpa-vits-bilingual");
```



```
    setTtsLangCode("auto");
    setTtsDevice("cpu");
  } else if (style === "kokoro") {
    setTtsVoice("zf_xiaoxiao");
    setTtsEnglishVoice("af_heart");
    setTtsModel("kokoro-82m");
    setTtsLangCode("z");
    setTtsDevice("auto");
  }
};

const ttsPayload = () => ({
  // 统一组装请求体，保存和预加载两条路径使用完全相同的设置快照。
  enabled: ttsEnabled,
  api_url: ttsUrl,
  api_style: ttsStyle,
  voice: ttsVoice,
  english_voice: ttsEnglishVoice,
  model: ttsModel,
  lang_code: ttsLangCode,
  device: ttsDevice,
  speed: clampSpeed(ttsSpeed),
});

const save = async (event) => {
  event.preventDefault();
  setBusy(true);
  setMessage("");
  try {
    const payload = ttsPayload();
    storeSpeed(payload.speed);

    // DeepSeek 与 TTS 设置互不依赖，并行保存可缩短用户等待时间。
    const [deepseek, tts] = await Promise.all([
      api.updateDeepSeekSettings({ api_key: apiKey, api_base: apiBase, model, mock }),
      api.updateTTSSettings(payload),
    ]);
    setMasked(deepseek.api_key_masked);

    // 保存后清空密钥明文，只保留掩码，降低浏览器内存中长期驻留的风险。
    setApiKey("");

    if (typeof tts.speed === "number") updateSpeed(tts.speed);

    setMessage("设置已保存。");
  } catch (err) {
    setMessage(err.message);
  } finally {
    setBusy(false);
  }
};

const preload = async () => {
  setPreloading(true);
  setMessage("");
  try {
    const payload = ttsPayload();
```

```
storeSpeed(payload.speed);

// 先落库再预加载，保证后端按界面当前选择的引擎、设备和音色初始化。
await api.updateTTSSettings(payload);

const data = await api.preloadTTS();

setMessage(data.detail || "语音模型已加载。");
} catch (err) {

  setMessage(err.message);
} finally {

  setPreloading(false);
}
};

const usingKokoro = ttsStyle === "kokoro";
const usingSherpa = ttsStyle === "sherpa";

return (
<div className="fixed inset-0 z-40 flex items-center justify-center bg-black/30 px-4">

<form className="max-h-[92vh] w-full max-w-2xl overflow-y-auto rounded-md bg-white p-5 shadow-2xl" onSubmit={sav...

<div className="mb-4 flex items-center justify-between">

<h2 className="flex items-center gap-2 text-lg font-semibold text-stone-950">

  <KeyRound size={18} />

  模型与语音设置

</h2>

<button type="button" className="rounded-md p-1 hover:bg-stone-100" onClick={onClose} title="关闭">

  <X size={18} />

</button>

</div>

<section className="border-b border-stone-100 pb-4">

<h3 className="mb-3 text-sm font-semibold text-stone-500">DeepSeek</h3>

<Field label="API Key">

<input

  className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-stone-900"

  value={apiKey}

  onChange={(event) => setApiKey(event.target.value)}

  placeholder={masked ? `已保存：${masked}`, 留空则不修改`:"sk-..."}

  type="password"

/>

</Field>

<Field label="API Base">

<input className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-stone-900"...

</Field>

<Field label="模型">

<input className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-stone-900"...

</Field>

<label className="flex items-center gap-2 text-sm text-stone-700">

<input type="checkbox" checked={mock} onChange={(event) => setMock(event.target.checked)} />

  使用 mock 模式，不调用真实 DeepSeek

</label>

</section>

<section className="mt-4">

<h3 className="mb-3 flex items-center gap-2 text-sm font-semibold text-stone-500">

  <Volume2 size={16} />
```

```

    语音朗读
  </h3>
  <label className="mb-3 flex items-center gap-2 text-sm text-stone-700">
    <input type="checkbox" checked={ttsEnabled} onChange={(event) => setTtsEnabled(event.target.checked)} />
    启用语音朗读
  </label>
  <Field label={ 阅读速度：${clampSpeed(ttsSpeed).toFixed(2)}x}>
    <div className="flex items-center gap-3">
      <input className="w-full accent-stone-900" type="range" min="0.2" max="2" step="0.05" value={ttsSpeed} o...
      <input className="h-9 w-20 rounded-md border border-stone-300 px-2 text-sm outline-none focus:border-sto...
    </div>
  </Field>
  <div className="grid gap-3 sm:grid-cols-2">
    <Field label="语音引擎">
      <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-stone-9...
        <option value="browser">浏览器自带 TTS， 响应最快</option>
        <option value="kokoro">Kokoro 本地模型， 音质更自然</option>
        <option value="sherpa">Sherpa 原生中英文， 轻量离线</option>
      </select>
    </Field>
    <Field label="推理设备">
      <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-stone-9...
        <option value="auto">auto： 优先 GPU</option>
        <option value="cuda">cuda： 使用 GPU</option>
        <option value="cpu">cpu： 使用 CPU</option>
      </select>
    </Field>
  </div>
  {usingKokoro && (
    <>
      <Field label="服务地址">
        <input className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-stone-...
      </Field>
      <div className="grid gap-3 sm:grid-cols-2">
        <Field label="中文 Voice">
          <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-sto...
            {CHINESE_VOICES.map((voice) => (
              <option key={voice.value} value={voice.value}>{voice.label}</option>
            ))}
          </select>
        </Field>
        <Field label="英文 Voice">
          <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-sto...
            {ENGLISH_VOICES.map((voice) => (
              <option key={voice.value} value={voice.value}>{voice.label}</option>
            ))}
          </select>
        </Field>
      </div>
    </div>
  </div className="grid gap-3 sm:grid-cols-2">
```

```
<Field label="语言模式">
  <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-sto...
    <option value="z">z：中文普通话模型</option>
    <option value="auto">auto：中英自动识别</option>
    <option value="a">a：美式英语</option>
    <option value="b">b：英式英语</option>
  </select>
</Field>
<Field label="Model">
  <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-sto...
    <option value="kokoro-82m">Kokoro-82M</option>
  </select>
</Field>
</div>
<button type="button" className="inline-flex h-9 items-center gap-2 rounded-md border border-stone-300 p...
  {preloading ? <Loader2 className="animate-spin" size={15} /> : <Volume2 size={15} />}
  加载语音模型
</button>
</>
)}
{usingSherpa && (
  <>
    <p className="mb-3 text-sm leading-6 text-stone-600">
      原生 ONNX CPU 推理。首次加载会下载中英文模型，之后完全离线运行并复用语音缓存。
    </p>
    <div className="grid gap-3 sm:grid-cols-2">
      <Field label="中文 Voice">
        <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-sto...
          {SHERPA_CHINESE_VOICES.map((voice) => (
            <option key={voice.value} value={voice.value}>{voice.label}</option>
          ))}
        </select>
      </Field>
      <Field label="英文 Voice">
        <select className="w-full rounded-md border border-stone-300 px-3 py-2 outline-none focus:border-sto...
          {SHERPA_ENGLISH_VOICES.map((voice) => (
            <option key={voice.value} value={voice.value}>{voice.label}</option>
          ))}
        </select>
      </Field>
    </div>
    <button type="button" className="inline-flex h-9 items-center gap-2 rounded-md border border-stone-300 p...
      {preloading ? <Loader2 className="animate-spin" size={15} /> : <Volume2 size={15} />}
      下载并加载 Sherpa 模型
    </button>
  </>
)}
</section>
{message && <p className="mt-4 rounded-md bg-stone-100 px-3 py-2 text-sm text-stone-700">{message}</p>}
<button className="mt-4 inline-flex h-10 items-center gap-2 rounded-md bg-stone-900 px-4 text-sm font-medium t...
```

```

    <Save size={16} />
    保存设置
  </button>
</form>
</div>
);
}
function Field({ label, children }) {
  return (
    <label className="mb-3 block">
      <span className="mb-1 block text-sm font-medium text-stone-700">{label}</span>
      {children}
    </label>
  );
}
function readStoredSpeed(fallback = 0.8) {
  // SSR 或测试环境没有 window，此时安全回退，不在模块导入阶段抛错。
  if (typeof window === "undefined") return clampSpeed(fallback);
  return clampSpeed(window.localStorage.getItem(SPEED_STORAGE_KEY) ?? fallback);
}
function storeSpeed(value) {
  if (typeof window === "undefined") return;
  window.localStorage.setItem(SPEED_STORAGE_KEY, String(clampSpeed(value)));
}
function clampSpeed(value) {
  // UI 与后端共同限制 0.2 至 2 倍；非法输入恢复 0.8，防止 NaN 进入控件。
  const parsed = Number(value);
  if (!Number.isFinite(parsed)) return 0.8;
  return Math.min(2, Math.max(0.2, parsed));
}
// ===== FILE: frontend/src/components/OutlineTree.jsx =====
import { ChevronDown, ChevronRight } from "lucide-react";
import { useMemo, useState } from "react";
export default function OutlineTree({ outline, activeSectionIndex, onJump }) {
  // 只有大纲数据变化时才重建树，滚动切换活动卡片不会重复执行层级解析。
  const tree = useMemo(() => buildOutlineTree(outline), [outline]);
  return (
    <nav className="space-y-1 p-3">
      {tree.map((node) => (
        <OutlineNode
          key={node.key}
          node={node}
          activeSectionIndex={activeSectionIndex}
          onJump={onJump}
        />
      ))}
    </nav>
  );
}
function OutlineNode({ node, activeSectionIndex, onJump }) {
```

```
// 每个节点独立保存展开状态， 用户可只折叠暂时不看的章节。
const [open, setOpen] = useState(true);
const hasChildren = node.children.length > 0;
const isActive = activeSectionIndex === node.index;
return (
  <div>
    <div className={flex items-start gap-1 rounded-md ${isActive ? "bg-stone-900 text-white" : "hover:bg-stone-100"} ...
      <button
        className="mt-1 inline-flex h-6 w-6 shrink-0 items-center justify-center rounded"
        onClick={() => setOpen(!open)}
        disabled={!hasChildren}
        title={hasChildren ? "展开/折叠" : ""}
      >
        {hasChildren ? (open ? <ChevronDown size={15} /> : <ChevronRight size={15} />) : <span className="h-4 w-4" /...
      </button>
      <button className="min-w-0 flex-1 px-1 py-2 text-left text-sm" onClick={() => onJump(node.index)}>
        <span className="block truncate font-medium">{node.title}</span>
        <span className={mt-1 block text-xs ${isActive ? "text-stone-200" : "text-stone-500"}}>
          {node.card_count} 张卡片
        </span>
      </button>
    </div>
    {hasChildren && open && (
      <div className="ml-4 border-l border-stone-200 pl-2">
        {node.children.map((child) => (
          <OutlineNode
            key={child.key}
            node={child}
            activeSectionIndex={activeSectionIndex}
            onJump={onJump}
          />
        ))}
      </div>
    )}
  </div>
);
}
```

```
function buildOutlineTree(outline) {
  // 数字标题如 2、2.1、2.1.3 按编号寻找父节点；Map 保存已经出现的编号节点。
  const roots = [];
  const byNumber = new Map();
  let lastTop = null;
  for (const item of outline) {
    const number = extractNumber(item.title);
    const node = { ...item, key: `${item.index}-${item.title}`, children: [], number };
    // 无编号小节挂在最近的顶级章节下；若尚无顶级章节，则自身成为根节点。
    if (!number) {
      if (lastTop) {
        lastTop.children.push(node);
      } else {
```

```
    roots.push(node);
    lastTop = node;
  }
  continue;
}
byNumber.set(number, node);
// 2.1.3 的直接父编号为 2.1；父项不存在时保持顶级，避免节点丢失。
const parentNumber = number.includes(".") ? number.split(".").slice(0, -1).join(".") : "";
const parent = parentNumber ? byNumber.get(parentNumber) : null;
if (parent) {
  parent.children.push(node);
} else {
  roots.push(node);
  lastTop = node;
}
}
return roots;
}

function extractNumber(title) {
  // 只识别标题开头的层级数字，正文中间出现的数字不会改变树结构。
  const match = String(title).match(/^\s*(\d+(?:\.\d+)*)\b/);
  return match?.[1] || "";
}

// ===== FILE: frontend/src/components/RichText.jsx =====
import katex from "katex";
import "katex/dist/katex.min.css";
export default function RichText({ text = "" }) {
  // 先解析为段落、列表和块公式，再分别渲染，避免直接输出模型返回的 HTML。
  const blocks = splitBlocks(text);
  return (
    <div className="space-y-3 text-sm leading-7 text-stone-800">
      {blocks.map((block, index) => {
        if (block.type === "math") {
          return <MathBlock key={index} latex={block.content} />;
        }
        if (block.type === "list") {
          return (
            <ul key={index} className="list-disc space-y-1 pl-5">
              {block.items.map((item, itemIndex) => (
                <li key={` ${item}-${itemIndex}`}>{renderInlineMath(item)}</li>
              ))}
            </ul>
          );
        }
        return <p key={index}>{renderInlineMath(block.content)}</p>;
      })}
    </div>
  );
}

function MathBlock({ latex }) {
```

```
// 块公式允许水平滚动，较长的推导不会撑破卡片宽度或覆盖相邻内容。

return (
  <div className="overflow-x-auto rounded-md bg-stone-50 px-3 py-3 text-center">
    <span dangerouslySetInnerHTML={{ __html: renderKatex(latex, true) }} />
  </div>
);
}

function splitBlocks(text) {
  // DeepSeek 可能返回 $...$, $$...$$、\(...\) 或 \[...\] 四种公式边界。
  // 先统一成美元符号格式，再区分块级公式、列表和普通段落，避免公式被当作纯文本换行。
  const normalized = String(text || "")
    .replace(/\r\n/g, "\n")
    .replace(/\[/g, "$$")
    .replace(/\]/g, "$$")
    .replace(/\(/g, "$")
    .replace(/\)/g, "$");
  const parts = normalized.split(/(\${\$|s\S}*\${\$})/g).filter(Boolean);
  const blocks = [];
  for (const part of parts) {
    if (part.startsWith("$") && part.endsWith("$")) {
      blocks.push({ type: "math", content: part.slice(2, -2).trim() });
      continue;
    }
    const paragraphs = part.split(/\n\s*\n/).map((item) => item.trim()).filter(Boolean);
    for (const paragraph of paragraphs) {
      const lines = paragraph.split("\n").map((line) => line.trim()).filter(Boolean);
      if (lines.length > 1 && lines.every((line) => /^[^*]\s+/.test(line))) {
        blocks.push({ type: "list", items: lines.map((line) => line.replace(/^[^*]\s+/, "")) });
      } else {
        blocks.push({ type: "paragraph", content: paragraph });
      }
    }
  }
  return blocks;
}

function renderInlineMath(text) {
  // 普通文字保持 React 文本节点，只有成对的单美元符号片段进入 KaTeX。
  const parts = String(text).split(/(\${^\$}\n+\$)/g).filter(Boolean);
  return parts.map((part, index) => {
    if (part.startsWith("$") && part.endsWith("$")) {
      return (
        <span
          key={index}
          className="mx-0.5"
          dangerouslySetInnerHTML={{ __html: renderKatex(part.slice(1, -1), false) }}
        />
      );
    }
    return <span key={index}>{part}</span>;
  });
}
```



```
}

function renderKatex(latex, displayMode) {

  try {

    // 禁止 KaTeX trust 模式， 模型返回的公式不能注入链接、HTML 或其他受信任命令。

    // throwOnError=false 让局部公式错误不会中断整张卡片；真正的异常再回退为转义后的原文。

    return katex.renderToString(latex, {

      displayMode,

      throwOnError: false,

      strict: false,

      trust: false,

    });

  } catch {

    return escapeHtml(latex);

  }

}

function escapeHtml(value) {

  // KaTeX 极端异常时回显经过转义的原公式，既保留可读信息又避免 HTML 注入。

  return String(value)

    .replace(/&/g, "&amp;")

    .replace(/</g, "&lt;")

    .replace(/>/g, "&gt;")

    .replace(/"/g, "&quot;");

}

// ===== FILE: frontend/src/components/UploadBox.jsx =====

import { Upload } from "lucide-react";

import { useRef } from "react";

export default function UploadBox({ busy, onUpload }) {

  // 隐藏原生文件输入框，由统一样式按钮触发；拖拽与选择最终调用同一上传函数。

  const inputRef = useRef(null);

  return (

    <div

      className="rounded-md border-2 border-dashed border-stone-300 bg-white p-6 shadow-sm"

      onDragOver={(event) => event.preventDefault()}

      onDrop={(event) => {

        event.preventDefault();

        // MVP 每次只处理第一个文件，批量导入留给后续版本。

        const file = event.dataTransfer.files?.[0];

        if (file) onUpload(file);

      }}

    >

    <input

      ref={inputRef}

      type="file"

      accept=".md,.txt"

      className="hidden"

      onChange={(event) => {

        const file = event.target.files?.[0];

        if (file) onUpload(file);

      }}

    />

  )

}
```

```
<button
  className="flex w-full flex-col items-center gap-3 rounded-md bg-stone-50 px-4 py-10 text-center hover:bg-ston...
  onClick={() => inputRef.current?.click()}
  disabled={busy}
>
<Upload size={28} />
<span className="font-semibold">{busy ? "上传中..." : "选择或拖拽 Markdown/TXT 文件"}</span>
<span className="text-sm text-stone-500">上传后会保存原文并按章节切分</span>
</button>
</div>
);
}

// ===== FILE: frontend/src/components/VoiceButton.jsx =====
import { Loader2, Volume2, VolumeX } from "lucide-react";
import { useRef, useState } from "react";
import { api } from "../api/client";
const MAX_REMOTE_CHARS = 5000;
const SPEED_STORAGE_KEY = "book-concept-app.tts.speed";
export default function VoiceButton({ card, text, label, compact = true, includeContext = true }) {
  // speaking 表示正在播放， loading 表示读取设置、加载模型或等待后端生成音频。
  // 两种状态分开后，按钮可准确显示“加载”与“停止”图标。
  const [speaking, setSpeaking] = useState(false);
  const [loading, setLoading] = useState(false);
  const audioRef = useRef(null);
  const objectUrlRef = useRef(null);
  const stoppedRef = useRef(false);
  const stop = () => {
    // 同时停止浏览器 TTS 和后端生成音频，并释放 Blob URL。
    // Blob URL 若不主动撤销会一直占用浏览器内存，连续朗读长寓言时尤其明显。
    stoppedRef.current = true;
    window.speechSynthesis?.cancel();
    audioRef.current?.pause();
    audioRef.current = null;
    if (objectUrlRef.current) {
      URL.revokeObjectURL(objectUrlRef.current);
      objectUrlRef.current = null;
    }
    setSpeaking(false);
    setLoading(false);
  };
  const speak = async () => {
    if (speaking || loading) {
      stop();
      return;
    }
    // 朗读副本依次经过上下文组装、LaTeX 简化、符号清洗和长度保护。
    // 页面显示文本完全不变，公式仍由 KaTeX 按原始内容渲染。
    const content = limitText(sanitizeForSpeech(stripLatex(buildReadableText(card, text, includeContext))));
    if (!content) return;
    stoppedRef.current = false;
```

```
setLoading(true);

try {
  // 每次朗读读取当前设置，确保用户刚调整的引擎和语速立即生效。
  // 远程或本地模型不可用时自动回退浏览器 TTS，让朗读按钮始终有可用路径。
  const settings = await api.getTTSSettings().catch(() => null);
  const speed = readStoredSpeed(settings?.speed ?? 0.8);
  if (settings?.api_style === "browser") {
    setLoading(false);
    browserSpeak(content, setSpeaking, speed);
    return;
  }
  // 预加载失败仍继续请求语音，服务端会在朗读入口再次尝试懒加载或返回明确错误。
  await api.preloadTTS().catch(() => null);
  await playRemoteWholeText(content, speed);
} catch {
  if (!stoppedRef.current) browserSpeak(content, setSpeaking, readStoredSpeed());
} finally {
  setLoading(false);
}

};

const playRemoteWholeText = async (content, speed) => {
  // 后端负责模型推理与磁盘缓存，前端只接收最终音频 Blob。
  // stoppedRef 用于处理用户在请求尚未返回时点击停止的竞态，避免迟到的响应突然开始播放。
  setSpeaking(true);
  const blob = await api.synthesizeSpeech(content, { speed });
  if (stoppedRef.current) return;
  const url = URL.createObjectURL(blob);
  objectUrlRef.current = url;
  await playAudioUrl(url);
  stop();
};

const playAudioUrl = (url) =>
  new Promise((resolve, reject) => {
    // 使用独立 Audio 对象播放 Blob URL，结束和错误都通过 Promise 回到统一清理流程。
    const audio = new Audio(url);
    audioRef.current = audio;
    audio.onended = resolve;
    audio.onerror = reject;
    audio.play().then(() => setLoading(false)).catch(reject);
  });

return (
  <button
    className={
      compact
      ? "inline-flex h-9 w-9 items-center justify-center rounded-md border border-stone-200 hover:bg-stone-50"
      : "inline-flex h-8 items-center gap-1 rounded-md border border-stone-200 px-2 text-xs hover:bg-stone-50"
    }
    onClick={speak}
    title={label || "朗读"}
  >
```

```

    {loading ? <Loader2 className="animate-spin" size={compact ? 18 : 15} /> : speaking ? <VolumeX size={compact ? 1...
    {!compact && <span>{label || "朗读"}</span>}
  }
</button>
);
}

function buildReadableText(card, sectionText, includeContext) {
  // 卡片主按钮默认朗读 “ 标题 + 一句话 + 当前段落 ” ；寓言按钮关闭上下文后只读寓言。
  if (!card) return sectionText || "";
  if (!includeContext) return sectionText || "";
  const current = sectionText || card.simple_explanation || card.fable || "";
  return [card.title, card.one_sentence, current].filter(Boolean).join("。");
}

function limitText(value) {
  // 对远程模型设置字符上限，避免超长卡片触发请求超时或过高内存占用。
  const content = String(value || "").trim();
  if (content.length <= MAX_REMOTE_CHARS) return content;
  return `${content.slice(0, MAX_REMOTE_CHARS)}。`;
}

function browserSpeak(content, setSpeaking, speed) {
  // 浏览器 TTS 作为低延迟模式和本地模型失败时的最终回退路径。
  if (!("speechSynthesis" in window)) return;
  const utterance = new SpeechSynthesisUtterance(content);
  utterance.lang = "zh-CN";
  utterance.rate = speed;
  utterance.pitch = 1.05;
  const voice = pickChineseVoice(window.speechSynthesis.getVoices());
  if (voice) utterance.voice = voice;
  utterance.onend = () => setSpeaking(false);
  utterance.onerror = () => setSpeaking(false);
  setSpeaking(true);
  window.speechSynthesis.cancel();
  window.speechSynthesis.speak(utterance);
}

function readStoredSpeed(fallback = 0.8) {
  // 每次点击朗读都重新读取持久化语速，设置调整无需重新挂载卡片组件。
  if (typeof window === "undefined") return clampSpeed(fallback);
  return clampSpeed(window.localStorage.getItem(SPEED_STORAGE_KEY) ?? fallback);
}

function clampSpeed(value) {
  const parsed = Number(value);
  if (!Number.isFinite(parsed)) return 0.8;
  return Math.min(2, Math.max(0.2, parsed));
}

function pickChineseVoice(voices) {
  // 优先系统自然中文音色；没有中文时再选择在线自然音色，最终允许浏览器默认。
  const chineseVoices = voices.filter((voice) => /zh|Chinese|Mandarin|普通话|中文/i.test(`${voice.lang} ${voice.name}`) ...
  return (
    chineseVoices.find((voice) => /Natural|Online|Microsoft|Xiaoxiao|Yunxi|Huihui|Yaoyao/i.test(voice.name)) ||
    chineseVoices[0] ||
    voices.find((voice) => /Natural|Online/i.test(voice.name)) ||
  )
}
```



```
const [bookData, cardData] = await Promise.all([api.getBook(bookId), api.listCards(bookId)]);

setBook(bookData);

setCards(cardData);

};

useEffect(() => {

  load().catch((err) => setError(err.message));

}, [bookId]);

const generate = async ({ force = false } = {}) => {

  setBusy(true);

  setError("");

  setMessage(force ? "正在清空旧卡片， 并从第一节重新生成..." : "正在按小节生成， 单次最多 20 张卡片...");

  try {

    // force 模式由后端删除旧体系；前端同步清除旧卡片对应的阅读位置。

    const data = await api.generateCards(bookId, { force });

    if (force) localStorage.removeItem(`reader-progress-${bookId}`);

    await load();

    setMessage(`${data.message}：本次新增 ${data.generated} 张， 进度 ${data.cursor}/${data.total_sections}。`);

  } catch (err) {

    setError(err.message);

    setMessage("");

  } finally {

    setBusy(false);

  }

};

if (!book) {

  return <div className="mx-auto max-w-4xl px-4 py-12 text-stone-500">加载中...</div>;

}

return (

  <section className="mx-auto max-w-5xl px-4 py-8">

    <div className="mb-6 flex flex-col justify-between gap-4 sm:flex-row sm:items-end">

      <div>

        <p className="text-sm text-stone-500">{book.filename}</p>

        <h1 className="mt-1 text-3xl font-bold text-stone-950">{book.title}</h1>

        <p className="mt-2 text-stone-600">

          当前已有 {cards.length} 张知识卡片， 已处理到第 {book.generation_cursor || 0} 个小节。

        </p>

      </div>

      <div className="flex flex-wrap gap-2">

        <button

          className="inline-flex h-10 items-center gap-2 rounded-md border border-stone-300 bg-white px-4 text-sm fo...

          onClick={() => generate({ force: false })}

          disabled={busy}

        >

          {busy ? <Loader2 className="animate-spin" size={16} /> : <Sparkles size={16} />}

          继续生成

        </button>

        <button

          className="inline-flex h-10 items-center gap-2 rounded-md border border-stone-300 bg-white px-4 text-sm fo...

          onClick={() => generate({ force: true })}

          disabled={busy}

        >
```

```

    title="清空旧卡片并从头生成体系卡"
  >
  <RefreshCw size={16} />
  从头生成
</button>
<button
  className="inline-flex h-10 items-center gap-2 rounded-md bg-stone-900 px-4 text-sm font-medium text-white...
  onClick={() => onRead(bookId)}
  disabled={!cards.length}
  title="跳转到上次退出时阅读的卡片"
>
  <Play size={16} />
  开始阅读
</button>
</div>
</div>
<div className="mb-4 rounded-md border border-blue-200 bg-blue-50 px-3 py-3 text-sm leading-6 text-blue-900">
  生成逻辑：按小节交给 DeepSeek。每个小节优先生成一张导览卡，说明这一节为什么需要、能做什么、会涉及哪些下级概念...
</div>
{message && <p className="mb-4 rounded-md border border-stone-200 bg-white px-3 py-2 text-sm text-stone-700">{me...
{error && (
  <p className="mb-4 flex items-start gap-2 rounded-md border border-red-200 bg-red-50 px-3 py-2 text-sm text-re...
  <AlertTriangle className="mt-0.5 shrink-0" size={16} />
  <span>{error}</span>
</p>
)}
<div className="grid gap-3 md:grid-cols-2">
  {/* 概览页只展示可扫描摘要，完整寓言、公式和关系在沉浸阅读页展开。 */}
  {cards.map((card) => (
    <article key={card.id} className="rounded-md border border-stone-200 bg-white p-4 shadow-sm">
      <p className="text-xs text-stone-500">
        {card.card_type === "section_overview" ? "导览" : "概念"} · {card.chapter}
      </p>
      <h2 className="mt-1 font-semibold text-stone-950">{card.title}</h2>
      <p className="mt-2 line-clamp-3 text-sm leading-6 text-stone-600">{card.one_sentence}</p>
    </article>
  )))}
</div>
</section>
);
}
// ===== FILE: frontend/src/pages/CardReaderPage.jsx =====
import { useEffect, useMemo, useRef, useState } from "react";
import { Loader2, Sparkles } from "lucide-react";
import { api } from "../api/client";
import ConceptCard from "../components/ConceptCard";
import ChatPanel from "../components/ChatPanel";
import OutlineTree from "../components/OutlineTree";
export default function CardReaderPage({ bookId }) {
  const [cards, setCards] = useState([]);
```

```
const [outline, setOutline] = useState([]);
const [active, setActive] = useState(0);
const [error, setError] = useState("");
const [prefetching, setPrefetching] = useState(false);
const [generationStatus, setGenerationStatus] = useState("");
const containerRef = useRef(null);
const cardRefs = useRef({});
const restoredRef = useRef(false);
const prefetchingRef = useRef(false);
const requestedCursorRef = useRef(null);
const progressKey = useMemo(() => `reader-progress-${bookId}`, [bookId]);
const autoPrefetchKey = useMemo(() => `reader-auto-prefetch-${bookId}`, [bookId]);
const [autoPrefetch, setAutoPrefetch] = useState(() => localStorage.getItem(autoPrefetchKey) !== "false");
const load = async () => {
  // 卡片与大纲并行刷新，生成下一节后一次更新阅读区和左侧进度状态。
  const [cardData, outlineData] = await Promise.all([api.listCards(bookId), api.getOutline(bookId)]);
  setCards(cardData);
  setOutline(outlineData);
  return { cardData, outlineData };
};
useEffect(() => {
  // 切换书籍时重置一次性恢复标记和预取防重标记，不能沿用上一册的状态。
  restoredRef.current = false;
  requestedCursorRef.current = null;
  setGenerationStatus("");
  setAutoPrefetch(localStorage.getItem(autoPrefetchKey) !== "false");
  load().catch((err) => setError(err.message));
}, [bookId, autoPrefetchKey]);
useEffect(() => {
  if (restoredRef.current || !cards.length) return;
  // 卡片首次加载完成后只恢复一次阅读位置。
  // 优先按 cardId 定位可抵抗后台新增卡片导致的索引变化；找不到旧卡片时再回退到保存的 index。
  restoredRef.current = true;
  const saved = readProgress(progressKey);
  const savedIndex = cards.findIndex((card) => card.id === saved.cardId);
  const nextActive = savedIndex >= 0 ? savedIndex : Math.min(saved.index || 0, cards.length - 1);
  setActive(nextActive);
  requestAnimationFrame(() => {
    cardRefs.current[cards[nextActive]?.id]?.scrollIntoView({ behavior: "auto", block: "start" });
  });
}, [cards, progressKey]);
useEffect(() => {
  const activeCard = cards[active];
  if (!activeCard) return;
  // 活动卡片变化时立即保存 cardId 和索引，关闭页面或重启应用后可以继续阅读。
  localStorage.setItem(
    progressKey,
    JSON.stringify({
      cardId: activeCard.id,
      index: active,
```



```
      sectionIndex: activeCard.section_index,
      savedAt: Date.now(),
    }},
  );
  if (autoPrefetch) {
    // 用户进入当前卡片后立即检查下一节，不再等待固定延时。
    // 生成任务在后台执行，左侧状态文字会同步展示正在处理的小节和整体进度。
    maybeGenerateNextSection(activeCard);
  }
}, [active, cards, outline, progressKey, autoPrefetch]);

const updateFavorite = (cardId, isFavorite) => {
  setCards((items) => items.map((item) => (item.id === cardId ? { ...item, is_favorite: isFavorite } : item)));
};

const jumpToSection = (sectionIndex) => {
  // 一个小节可能包含多张卡，点击大纲时定位该节第一张卡作为入口。
  const target = cards.find((card) => card.section_index === sectionIndex);
  if (!target) return;
  cardRefs.current[target.id]?.scrollIntoView({ behavior: "smooth", block: "start" });
};

const toggleAutoPrefetch = (checked) => {
  setAutoPrefetch(checked);
  localStorage.setItem(autoPrefetchKey, checked ? "true" : "false");
};

const maybeGenerateNextSection = async (activeCard = cards[active]) => {
  // prefetchingRef 防止同一组件内并发请求，requestedCursorRef 防止状态更新期间重复请求同一节。
  // 两层保护与后端的小节 claim 机制配合，即使快速上下滑动也只会生成一份后续内容。
  if (!activeCard || prefetchingRef.current) return;
  // 复制后排序，避免原地 sort 改写 React state 并造成不可预测的重渲染。
  const ordered = outline.slice().sort((a, b) => a.index - b.index);
  const currentOutlineIndex = ordered.findIndex((item) => item.index === activeCard.section_index);
  if (currentOutlineIndex < 0) return;
  const next = ordered[currentOutlineIndex + 1];
  if (!next || next.generated || requestedCursorRef.current === next.index) return;
  requestedCursorRef.current = next.index;
  prefetchingRef.current = true;
  setPrefetching(true);
  setGenerationStatus(`正在生成第 ${currentOutlineIndex + 2} / ${ordered.length} 节：${next.title}`);
  try {
    // 后端游标决定真正处理的小节，前端的 next 仅用于去重和显示可读进度。
    const result = await api.generateCards(bookId, { force: false });
    setGenerationStatus(`已新增 ${result.generated} 张卡片，进度 ${result.cursor} / ${result.total_sections}`);
    await load();
  } catch (err) {
    requestedCursorRef.current = null;
    setGenerationStatus(`生成失败：${err.message}`);
  } finally {
    prefetchingRef.current = false;
    setPrefetching(false);
  }
};
```

```
if (error) {
  return <div className="mx-auto max-w-4xl px-4 py-12 text-red-700">{error}</div>;
}

const activeCard = cards[active];

return (
  <section className="grid h-[calc(100vh-56px)] lg:grid-cols-[280px_minmax(0,1fr)_420px]">
    <aside className="hidden h-[calc(100vh-56px)] overflow-y-auto border-r border-stone-200 bg-white lg:block">
      <div className="sticky top-0 border-b border-stone-100 bg-white p-4">
        <h2 className="font-semibold text-stone-950">文章大纲</h2>
        <p className="mt-1 text-xs text-stone-500">点击小节跳到对应卡片</p>
        <div className="mt-3 flex flex-col gap-2">
          <button
            className="inline-flex h-8 items-center justify-center gap-1 rounded-md border border-stone-300 px-2 tex...
            onClick={() => maybeGenerateNextSection()}
            disabled={prefetching || !activeCard}
          >
            {prefetching ? <Loader2 className="animate-spin" size={13} /> : <Sparkles size={13} />}
            生成下一节
          </button>
          <label className="flex items-center gap-2 text-xs text-stone-600">
            <input type="checkbox" checked={autoPrefetch} onChange={(event) => toggleAutoPrefetch(event.target.check...
            阅读时立即自动生成下一节
          </label>
        </div>
        {(prefetching || generationStatus) && (
          <p className={`mt-2 inline-flex items-start gap-1 rounded-md px-2 py-1 text-xs ${prefetching ? "bg-blue-50...
            {prefetching && <Loader2 className="mt-0.5 animate-spin" size={13} />}
            {generationStatus}
          </p>
        )}
      </div>
      <OutlineTree outline={outline} activeSectionIndex={activeCard?.section_index} onJump={jumpToSection} />
    </aside>
    <div
      ref={containerRef}
      className="reader-scroll h-[calc(100vh-56px)] overflow-y-auto bg-[#e8ece6]"
      onScroll={(event) => {
        // 每张卡固定占满一个阅读视口，因此滚动距离除以容器高度即可得到当前卡片索引。
        // 四舍五入让卡片越过半屏时才切换活动状态，减少边界附近反复触发后台预生成。
        const index = Math.round(event.currentTarget.scrollTop / event.currentTarget.clientHeight);
        setActive(Math.min(Math.max(index, 0), Math.max(cards.length - 1, 0)));
      }}
    >
      {cards.map((card, index) => (
        <div
          key={card.id}
          ref={(node) => {
            if (node) cardRefs.current[card.id] = node;
          }}
          className="reader-card flex h-[calc(100vh-56px)] items-center justify-center px-4 py-6"
```

```
>
    <ConceptCard card={card} index={index} total={cards.length} onFavorite={updateFavorite} />
  </div>
))}
{!cards.length && <div className="p-8 text-stone-600">还没有知识卡片，请先生成。</div>}
</div>
<ChatPanel card={activeCard} />
</section>
);
}
function readProgress(key) {
  // localStorage 可能被手工修改或留下旧格式；解析失败时按无历史进度处理。
  try {
    return JSON.parse(localStorage.getItem(key) || "{}");
  } catch {
    return {};
  }
}
// ===== FILE: frontend/src/pages/UploadPage.jsx =====
import { useState } from "react";
import { FileText } from "lucide-react";
import { api } from "../api/client";
import UploadBox from "../components/UploadBox";
export default function UploadPage({ books, onUploaded, onOpenBook }) {
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState("");
  const upload = async (file) => {
    // busy 同时禁止重复选择文件；无论成功失败都在 finally 中恢复可操作状态。
    setBusy(true);
    setError("");
    try {
      const data = await api.uploadBook(file);
      // 上传成功后由父组件刷新书库并跳转书籍详情，不在本页复制导航状态。
      onUploaded(data.book_id);
    } catch (err) {
      setError(err.message);
    } finally {
      setBusy(false);
    }
  };
  return (
    <section className="mx-auto grid min-h-[calc(100vh-56px)] max-w-6xl gap-6 px-4 py-8 lg:grid-cols-[1.05fr_0.95fr]">
      <div className="flex flex-col justify-center gap-5">
        <div>
          <h1 className="text-3xl font-bold tracking-normal text-stone-950 sm:text-4xl">上传一本书，生成可追问的概念卡...
          <p className="mt-3 max-w-2xl text-base leading-7 text-stone-600">
            支持 Markdown 和 TXT。系统会切分章节、提取核心概念，并生成解释、寓言、公式、前置知识和推荐问题。
          </p>
        </div>
        <UploadBox busy={busy} onUpload={upload} />
      </div>
    </section>
  );
}
```

```
{error && <p className="rounded-md border border-red-200 bg-red-50 px-3 py-2 text-sm text-red-700">{error}</p>...
</div>
<aside className="py-2">
  <h2 className="mb-3 text-sm font-semibold uppercase tracking-normal text-stone-500">已上传书籍</h2>
  <div className="space-y-3">
    {books.map((book) => (
      <button
        key={book.id}
        className="flex w-full items-center justify-between rounded-md border border-stone-200 bg-white p-4 text...
        onClick={() => onOpenBook(book.id)}
      >
        <span className="flex min-w-0 items-center gap-3">
          <FileText className="shrink-0 text-stone-500" size={20} />
          <span className="min-w-0">
            <span className="block truncate font-semibold text-stone-900">{book.title}</span>
            <span className="text-sm text-stone-500">{book.file_type} · {book.card_count} 张卡片</span>
          </span>
        </span>
      </button>
    ))}
    {books.length && <p className="rounded-md border border-dashed border-stone-300 p-4 text-sm text-stone-500"...
  </div>
</aside>
</section>
);
}

// ===== FILE: frontend/src/styles/index.css =====
@tailwind base;
@tailwind components;
@tailwind utilities;

:root {
  color-scheme: light;
}

body {
  margin: 0;
  min-width: 320px;
  background: #f7f5ef;
  color: #17202a;
}

button,
input,
textarea {
  font: inherit;
}

.reader-scroll {
  scroll-snap-type: y mandatory;
}

.reader-card {
  scroll-snap-align: start;
}
```